

Filtering and Accelerating: A Unified Framework for High-Performance Persistence Estimation

Qilong Shi , Lu Cao , Weiqiang Xiao , Nianfu Wang , Wenjun Li , Tong Yang , *Member, IEEE*,
Zhijun Li , Weizhe Zhang , *Senior Member, IEEE*, and Mingwei Xu 

Abstract—Efficient data stream processing, particularly for persistence estimation, is crucial in handling high-velocity data streams characterized by skewed distributions of item frequencies. Unlike more straightforward frequency metrics, persistence captures items' recurrence across multiple time windows, posing a significant challenge to existing single-structure sketches where high-persistence and low-persistence items collide. To address this, we introduce the Hypersistent Sketch, a unified framework for high-performance estimation built on two decoupled mechanisms: filtering and accelerating. The filtering component, a Cold Filter, directly addresses the skewed nature of data streams. It separates hot items from the majority of cold ones, which allows for differential treatment. The accelerating component, a Burst Filter, then optimizes the processing of hot items. It significantly improves throughput by preventing repeated insertions within a single window. We demonstrate its generality by applying it to various state-of-the-art sketches (e.g., On-Off, Waving, P-Sketch), showing it consistently enhances their original performance. We also deploy our framework on Redis platforms, demonstrating the framework's broad applicability and scalability.

Index Terms—Data stream, persistence estimation, sketch.

Received 11 July 2025; revised 24 March 2026; accepted 9 May 2026. Date of publication 22 May 2026; date of current version 6 July 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62221003, Grant 62472122, and Grant 62102203, in part by NSFC/RGC Collaborative Research Scheme under Grant 62461160332 and Grant CRS_HKUST602/24, in part by the Major Key Project of Peng Cheng Laboratory under Grant PCL2025A07, in part by the Young Top-notch Talent Project of Guangdong Province under Grant 2023TQ07X362, in part by Shenzhen Science and Technology Program under Grant ZDCY20250901101035012, and in part by the 2025 Research and Development of Security Detection and Defense Software for AI Models project from China Unicom Intelligent Manufacturing Technology & Industry (Guangdong) Company, Ltd. under Grant YGS25AE1000001. An earlier version of this paper was presented in part at the 2025 IEEE International Conference on Data Engineering [DOI: 10.1109/ICDE65448.2025.00227]. Recommended for acceptance by S. Shang. (Qilong Shi and Lu Cao contributed equally to this work.) (Corresponding authors: Wenjun Li; Weizhe Zhang; Mingwei Xu.)

Qilong Shi and Mingwei Xu are with the Department of Computer Science and Technology, Tsinghua University, Beijing 100871, China (e-mail: sql23@mails.tsinghua.edu.cn; xumw@tsinghua.edu.cn).

Lu Cao, Weiqiang Xiao, Nianfu Wang, and Weizhe Zhang are with the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen 518055, China, and also with Pengcheng Laboratory, Shenzhen 518055, China (e-mail: 22B951038@stu.hit.edu.cn; 24S151072@stu.hit.edu.cn; 23B951054@stu.hit.edu.cn; wzzhang@hit.edu.cn).

Wenjun Li is with Pengcheng Laboratory, Shenzhen 518055, China (e-mail: wenjunli@pku.org.cn).

Tong Yang is with the School of Computer Science, Peking University, Beijing 100871, China (e-mail: yangtong@pku.edu.cn).

Zhijun Li is with the School of Cyberspace Science, Harbin Institute of Technology, Harbin 150000, China (e-mail: lizhijun_os@hit.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TKDE.2026.3695579>, provided by the authors.

Digital Object Identifier 10.1109/TKDE.2026.3695579

I. INTRODUCTION

IN THE realm of data stream processing, an important characteristic – persistence, has received growing attention. Given an item e and a data stream with T non-overlapping and contiguous time windows, the persistence of e is defined as the number of time windows where e appears. The persistent items—those that appear consistently over time—are paramount [2], [3], [4], [5]. Their identification and quantification are critical across various applications, from security-related tasks such as intrusion detection and fraud prevention to operational enhancements in sectors like transportation analytics [6], [7], [8]. For example, persistent network threats may be strategically deployed at low frequencies to evade traditional detection systems, posing serious security risks [9], [10]. Similarly, in digital advertising, sophisticated schemes utilize automated scripts for click fraud, repeatedly engaging with ads to illegitimately boost revenue. Beyond malicious activities [11], [12], understanding item's persistence is also invaluable in identifying regular patterns [13], [14], [15], [16], [17], [18], optimizing operations [19], [20], and improving decision-making processes in dynamic environments like traffic flow management [21], [22], [23].

To bolster these applications in persistence estimation, one-pass Sketch algorithms are seen as particularly promising due to their efficiency and effectiveness [24], [25]. Recent advancements have focused on refining Sketch algorithms in various domains, including memory usage [26], [27], accuracy [28], [29], and processing speed [30], [31]. Yet, these enhancements often struggle to balance the trade-offs between memory usage and accuracy, a critical aspect given the increasing volume of data and the constraints of limited computational and memory resources [32], [33]. Taking the well-known On-Off Sketch [34] as an example, its primary data structure is similar to that of the traditional frequency estimation algorithm, the Count-Min Sketch, which consists of multiple columns of fixed-size counter arrays. In real-world scenarios, data streams exhibit high skewness, meaning the persistence of most items is relatively low (e.g., ≤ 5 , termed as cold items), while a minority of items show high persistence (referred to as hot items) [35], [36], [37], [38]. To accurately capture the persistence of hot items, the On-Off Sketch must allocate larger memory blocks (e.g., 32-bit counters) for each counter. This approach, however, leads to substantial memory wastage as many counters (those capturing cold items) do not utilize their higher bits [39], [40], [41].

Although several prior sketches adopt layered, hybrid, or bucket-based designs, their core goal is still to build a *standalone sketch* for a specific task within a single integrated structure. By contrast, Hypersistent Sketch is proposed as a *general enhancement framework* for persistence-related tasks. Its key idea is to explicitly separate cold items from hot candidates before they

enter the final persistence sketch, and then recover the throughput loss of this layered design through a persistence-specific Burst Filter. Therefore, the novelty of Hypersistent Sketch is not merely that it is multi-stage, but that it introduces a task-specific combination of *cold/hot separation* and *window-aware delayed update* into persistence sketching.

The memory-accuracy trade-off in existing single-structure sketches raises a critical question: Is it possible to design a unified framework that can be prepended to existing sketches to enhance their performance? In response, we propose the Hypersistent Sketch, a framework for high-performance estimation built upon two decoupled mechanisms: filtering and accelerating. The design of the Hypersistent Sketch framework is composed of three key components:

- 1) *Pluggable Hot Part for Core Estimation*: The core of the estimation is handled by the Hot Part. Critically, this is not a fixed structure but a slot where any state-of-the-art persistence sketch (e.g., On-Off Sketch, Waving Sketch) can be plugged in. Our framework enhances existing algorithms rather than replacing them entirely.
- 2) *Cold Filter for Filtering*: It addresses the item-collision issue by separating low-persistence (cold) items from high-persistence (hot) ones. This allows hot items to be processed by the main underlying sketch (i.e., the Hot Part), while cold items are handled with smaller, memory-efficient counters. This differential treatment significantly improves memory efficiency and accuracy.
- 3) *Burst Filter for Acceleration*: To counteract the potential processing overhead introduced by the Cold Filter, the Burst Filter optimizes the processing of hot items. Observing that an item's persistence can only increase by one per window, this filter intercepts repeated appearances of an item, deferring a single update to the subsequent structures until the window's end. This strategy dramatically increases throughput.

To demonstrate the framework's effectiveness and generality, we applied the Hypersistent Sketch to a wide array of state-of-the-art sketches, including the On-Off Sketch, Waving Sketch, P-Sketch, and Tight Sketch. The results consistently show that our framework enhances the original performance of each sketch, significantly reducing error rates while simultaneously increasing throughput. Furthermore, to showcase its broad applicability and scalability, we deployed the framework not only on CPU platforms but also within Redis, demonstrating its real-world feasibility and scalability.

Contributions and Organization. The main contributions of this paper are summarized as follows:

- We propose *Hypersistent Sketch*, a unified framework that can be prepended to existing persistence sketches to systematically enhance their performance.
- We design a *Cold Filter* to separate cold and hot items for more memory-efficient and accurate persistence estimation.
- We design a *Burst Filter* to suppress repeated per-window updates and thus significantly improve insertion throughput.
- We validate the proposed framework through extensive experiments and a Redis deployment, demonstrating its generality, effectiveness, and scalability.

The rest of the paper is organized as follows: Section II provides an overview of the background. Section III provides a recent development in persistence estimation.

Section IV describes the Hypersistent Sketch's data structure and algorithm. Section V offers a mathematical analysis of the algorithm, and Section VI presents the experimental results. Finally, the paper concludes with a summary in Section VII.

II. PROBLEM STATEMENT

• *Data Stream Model*: Consider a time-sequenced data stream S , represented as a set of tuples $\{(e_1, t_1), (e_2, t_2), \dots, (e_N, t_N)\}$, where each tuple (e_i, t_i) consists of a data item identifier e_i (such as a quintuple or source IP in network packets) and its arrival time t_i (the t_i is monotonically increasing). The set of data items E is defined as $E = \{e \mid \exists t \text{ such that } (e, t) \in S\}$.

• *Window*: The time range $[t_1, t_N]$ of the data stream S is evenly divided into w time windows, each of size $R = (t_N - t_1)/w$. These windows are represented as a set $W = \{[t_1, t_1 + R], [t_1 + R, t_1 + 2R], \dots, [t_1 + (w-1)R, t_N]\}$.

• *Persistence*: For a data stream S , the persistence of an item e_i is defined as the number of windows in which it appears, denoted p_{e_i} . It is formally defined as:

$$p_{e_i} = |\{[l, r] \mid (e_i, t) \in S \text{ and } t \in [l, r]\}|.$$

• *Persistence Estimation*: For all items in E , the task is to estimate their persistence.

• *Finding Persistent Item*: We define a threshold α , where an item e_i is considered α -persistent if its persistence $p_{e_i} \geq \alpha \cdot w$, that is, it appears in at least α times the total number of windows (the maximum persistence). The goal is to find all such α -persistent items. Here, α is not an internal hyperparameter, but a user-specified *application threshold* in the definition of the *finding persistent items* task. It directly reflects the persistence level of interest in the target application. For example, if the stream is divided into $w = 1000$ windows and $\alpha = 0.2$, then the goal is to report items that appear in at least 200 windows. Therefore, a smaller α corresponds to a broader notion of persistence and yields a larger candidate set, while a larger α focuses only on highly persistent items.

III. RELATED WORK

This subsection reviews work related to estimating persistence in data streams, a concept closely related to frequency estimation. The primary distinction between these two lies in their counting approach: while frequency estimation aggregates every occurrence of an item, persistence estimation counts an item only once per window, regardless of how many times it appears within that window.

This difference has led to innovative adaptations of existing frequency estimation algorithms [25], [42], [43], [44], [45]. One straightforward approach is to modify a frequency estimation Sketch by incorporating a deduplication step within each window, ensuring each item updates the Sketch's counter at most once per window. The PIE (Persistence via Incremental Estimation) algorithm [35] exemplifies this approach by combining a Count-Min (CM) Sketch [46] for frequency estimation with a Bloom Filter (BF) [47] for window-based deduplication. Here's how it works:

PIE Algorithm: As shown in Fig. 1, at the start of each window, when an item e arrives, it is hashed by d_1 hash functions to d_1 bits in a Bloom Filter. If any of these d_1 bits is 0, indicating e has not yet appeared in the current window, the bits are set to 1,

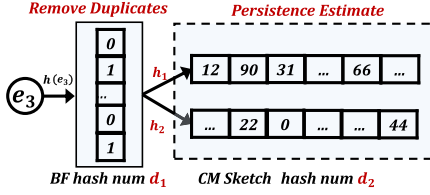


Fig. 1. Insertion of the strawman solution.

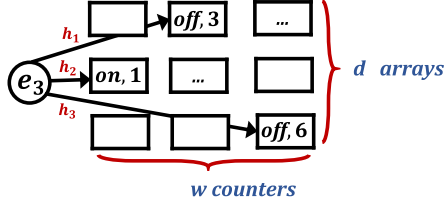


Fig. 2. An illustration of the On-Off Sketch framework.

and e is added to the CM Sketch on the right. The CM Sketch consists of d_2 rows of arrays, each containing w counters. The item e is hashed by d_2 different hash functions, each mapping to one counter per row, and these counters are incremented. If all d_1 bits in the Bloom Filter are 1, it indicates that e has already been accounted for in the current window, and no further action is taken.

When querying the persistence of an item e_2 , the algorithm checks the CM Sketch and returns the minimum value among the counters e_2 is mapped to by its d_2 hash functions.

Limitations: The PIE method has its shortcomings. First, the Bloom Filter is prone to false positives, which may report that an item has appeared when it has not. This can prevent a new item from being added to the CM Sketch, leading to underestimating its persistence. Additionally, hash collisions in the CM Sketch can cause counters to be incremented multiple times within a single window, even though an item's persistence should increase by only one per window. This results in a significant overestimation of persistence.

The well-known algorithm, the On-Off Sketch [34], made several improvements to address the inherent problems observed in previous approaches. There are two versions of the On-Off Sketch: one for persistence estimation (Version 1), which does not store item IDs, and another for finding persistent items (Version 2), which does store item IDs. Below, we discuss each version.

On-Off Sketch Version 1: As shown in Fig. 2, this version resembles the structure of a CM Sketch but adds a one-bit flag to each counter to indicate its state as either “on” or “off”. This flag determines whether the counter can be updated. When an item e arrives, it is mapped by d hash functions to d counters. Only those counters that are “on” are updated, after which their flags are set to “off”. At the end of each window and the start of a new one, all flags are reset to “on”. This mechanism ensures that within a window, a counter is incremented at most once, aligning with the definition of persistence and significantly reducing the overestimation.

On-Off Sketch Version 2: This version consists of an array of buckets, each containing multiple cells, as shown in Fig. 3. Each

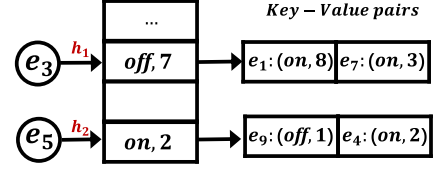


Fig. 3. On-Off Sketch for finding persistent items.

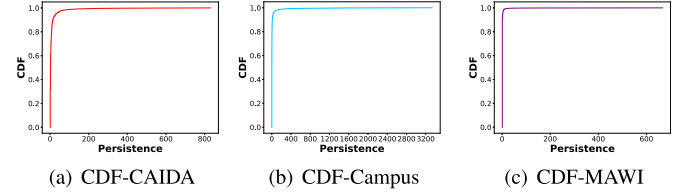


Fig. 4. Persistence variation across datasets.

cell has the structure $\langle ID, flag, counter \rangle$, and each bucket has a global cell $\langle flag, counter \rangle$. When item e arrives:

- 1) If e is already in the bucket, its counter is updated based on the on/off flag.
- 2) If e is not in the bucket and there is an empty cell, e is added as $\langle e, off, 1 \rangle$.
- 3) If e is not in the bucket and the bucket is full, the global cell is updated. If the global cell's counter exceeds the minimum counter in the bucket's cells, e is swapped into a bucket cell.

Limitations: Both versions do not account for the high skewness typical of natural data streams, where only a few items (hot items) have high persistence, while the majority (cold items) have persistence less than 5 (As shown in Fig. 4, the CDF plots of persistence for the three datasets validate this point). Without prior knowledge of which items are hot, all counters must be large enough (e.g., 32-bit) to accommodate potential hot items without risk of overflow, leading to low memory utilization. Additionally, in Version 2, many cold items and resulting hash collisions can cause frequent swaps between the global counter and bucket cells, severely overestimating persistence. This highlights the motivation behind our design: to separate cold and hot items within the data stream and handle them differently, which could greatly improve memory efficiency and reduce errors without sacrificing speed.

Beyond PIE and On-Off Sketch, several more recent sketch designs have also pushed forward persistence-related stream processing. WavingSketch [48] is a generic unbiased sketch for top- k item mining and has been shown applicable to persistent-item detection as well. Its main advantage is generality, but it is not specifically designed for persistence estimation, and thus does not directly target accurate whole-item persistence tracking over all items. P-Sketch [49] is a dedicated sketch for persistent item lookup, which protects candidate persistent items through probabilistic replacement based on both persistence and hotness. While effective for threshold-based persistent item detection, it is mainly tailored to lookup rather than to full persistence estimation. Tight-Sketch [50] generalizes this design philosophy to a wider spectrum of heavy-item-oriented tasks by using stochastic decay and compact bucket-based management. However, it still follows the route of designing a standalone sketch for heavy/persistent item detection tasks, instead of providing

TABLE I
MAIN SYMBOLS USED IN SECTION IV

Symbol	Meaning
L_1, L_2	Two arrays of the Cold Filter.
L_3	Bucket array of the Hot Part.
$f(\cdot)$	Hash function of the Burst Filter.
$h_i(\cdot), g_i(\cdot)$	Hash functions of L_1 and L_2 , respectively.
$H(\cdot)$	Hash function mapping an item to a bucket in L_3 .
d_1, d_2	Numbers of hash functions in L_1 and L_2 .
Δ_1, Δ_2	Promotion thresholds from L_1 to L_2 , and from L_2 to the Hot Part.
λ	Number of buckets in the Hot Part.
β	Number of cells in each Hot Part bucket.
γ	Number of cells in each Burst Filter bucket.
$key, per, flag$	ID field, persistence field, and per-window update flag.
$L_3[H(e)]$	The bucket selected for item e in the Hot Part.

a generic enhancement layer for existing persistence sketches. Stable-Sketch [51] further improves this family by exploiting multidimensional information, including item statistics and bucket stability, to achieve strong accuracy and high processing speed. Nevertheless, like P-Sketch and Tight-Sketch, it remains a monolithic sketch design mainly targeting heavy/persistent item detection rather than a unified framework for enhancing different persistence sketches. Therefore, although these methods substantially improve the state of the art for specific tasks, they do not address the same problem as ours, namely, how to *systematically strengthen a broad class of existing persistence sketches* through a shared front-end framework.

It is worth noting that, although some prior methods also contain multiple components or layered maintenance mechanisms, they are still essentially monolithic sketch designs for specific tasks. In contrast, Hypersistent Sketch is conceptually different: it is designed as a unified front-end enhancement framework that can be instantiated on top of different persistence sketches, rather than as another standalone sketch competing with them.

IV. HYPERSISTENT SKETCH DESIGN

This section details the design of the Hypersistent Sketch framework. We begin by presenting its overall architecture, key data structures, and operational workflow. To clarify its mechanics, we then walk through a concrete running example to illustrate the process. Finally, we discuss potential optimizations for the framework.

Notation and Main Data Structures: Before presenting the three stages of Hypersistent Sketch in detail, we first summarize the main symbols used throughout this section. The framework consists of the Burst Filter (Stage 1), the Cold Filter (Stage 2), and the Hot Part (Stage 3). The Cold Filter contains two arrays, denoted by L_1 and L_2 , while the Hot Part is organized as a bucket array L_3 . Thus, $L_3[H(e)]$ in Algorithm 1 denotes the bucket selected for item e in the Hot Part. Table I summarizes the key symbols.

A. Algorithm Framework Overview

As illustrated in Fig. 5, the Hypersistent Sketch framework is composed of three key components: a Burst Filter for acceleration, a Cold Filter for item separation, and a pluggable Hot Part that serves as the core estimator. A key design principle is that the Hot Part can be instantiated with any existing persistence sketch. To best explain our design rationale, we will introduce these components in the reverse order of data item (from Hot Part

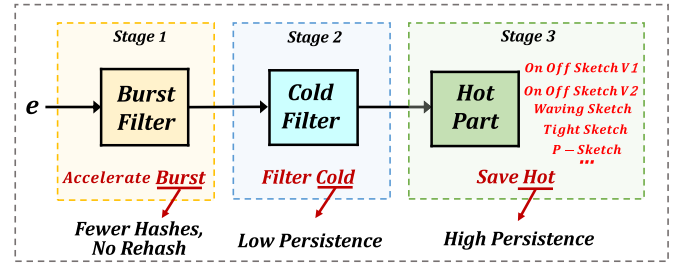


Fig. 5. Algorithm framework of Hypersistent Sketch.

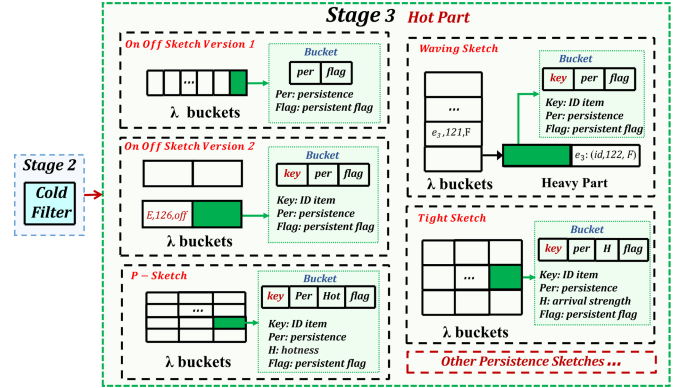


Fig. 6. Data structure of the Hot Part.

to Burst Filter), mirroring the sequence in which the framework was conceptually developed.

B. Hot Part (Stage 3)

The Hot Part, whose general structure is illustrated in Fig. 6, serves as the core estimator within the Hypersistent Sketch framework. As previously established, a key principle of our design is that the Hot Part is a pluggable component. It can be instantiated with any state-of-the-art persistence sketch, such as the On-Off Sketch, P-Sketch, or Waving Sketch. For the sake of clarity, we will use **a slightly modified version of the On-off Sketch** as an example to illustrate the operation.

In this example, the Hot Part consists of λ buckets, each containing β cells. Each entry stores an item's ID, a persistence value, and an on-off flag indicating whether the entry has been updated in the current time window (as each entry can be updated at most once within a single window). When an item E arrives, it is hashed into the $H(E)$ -th bucket $B[H(E)]$ via the hash function $H(\cdot)$. The cells in the bucket are then sequentially checked under the following three cases:

- 1) *If $B[H(E)]$ contains E :* If the flag is on, the persistence value is incremented by 1, and the flag is set to off. If the flag is off, no operation is performed.
- 2) *If $B[H(E)]$ does not contain E but has an empty entry:* The item $\langle E, 1, \text{off} \rangle$ is inserted into the empty entry.
- 3) *If $B[H(E)]$ neither contains E nor has any empty cells:* The entry with the smallest persistence value is probabilistically replaced, following the replacement strategy outlined in Algorithm 4 in appendix [52].

• **Limitations:** Our analysis reveals a fundamental limitation inherent in all these existing persistence sketches: their single-structure design. In such a monolithic architecture,

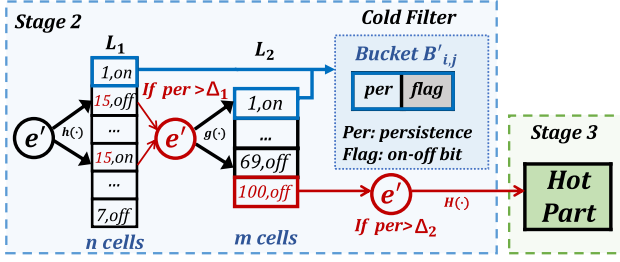


Fig. 7. Data structure of the Cold Filter.

high-persistence (hot) items and the far more numerous low-persistence (cold) items are forced to collide, competing for the same limited resources. Given the highly skewed persistence distribution in real-world data streams (Fig. 4), this design leads to a critical problem: nascent hot items are frequently and prematurely evicted by a deluge of cold items. This systemic collision significantly degrades the accuracy of identifying truly persistent items.

A straightforward and intuitive solution to address these challenges is to separate hot and cold items and store them independently. This is precisely the approach implemented by the Cold Filter in Stage 2.

C. Cold Filter (Stage 2)

Fig. 7 illustrates the structure of the Cold Filter. It consists of several arrays (two in this paper), L_1 and L_2 , where each cell in the arrays comprises a persistence counter and an on-off flag. When an item e' arrives, it is first hashed into multiple cells in L_1 (two in the example shown). We adopt an update strategy similar to the CU Sketch [53]: among the hashed cells with the flag set to on, the one with the smallest persistence value is selected. The processing follows two cases:

- 1) If the persistence value does not exceed a threshold Δ_1 , the persistence value is incremented by 1, and the flag is set to off.
- 2) If the persistence value exceeds Δ_1 , e' is inserted into L_2 using a similar strategy as for L_1 .

When inserting into L_2 , if the minimum persistence value of the cells mapped by e' in L_2 exceeds another threshold Δ_2 , e' is considered a hot item with high persistence. It is then inserted into the subsequent Hot Part, as described in the previous subsection. This design retains cold items within L_1 and L_2 , while only hot items proceed to the Hot Part. This separation effectively addresses the limitations mentioned in the previous subsection by clearly distinguishing between cold and hot items. The pseudocode for this process is presented in Algorithm 5 in appendix [52].

• **Limitations:** Although the proposed design addresses the separation of hot and cold items, and improves the accuracy of persistence estimation, it introduces a new challenge: additional hash computations are required to map items into the corresponding cells of L_1 and L_2 . Experiments (as shown in Section VI) indicate that using two hash functions for each layer yields optimal results. In contrast, the Hot Part only requires a single hash computation. Consequently, introducing the Cold Filter greatly reduces the algorithm's throughput.

To address this issue, we leverage a unique property of persistence estimation: **each cell in each array (L_n) can be updated at most once within a single window.** This means that for a

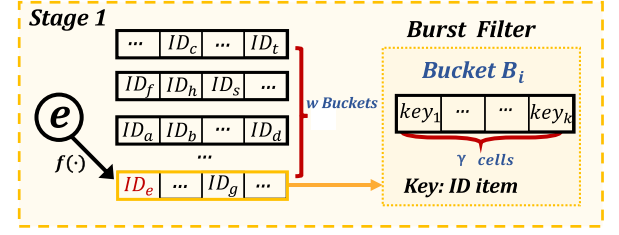


Fig. 8. Data structure of the Burst Filter.

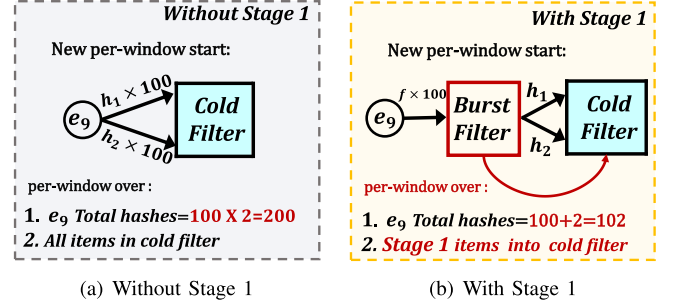


Fig. 9. Burst Filter for hash reduction.

given item e , only its first occurrence in a window is meaningful (as it updates the cells it maps to), while subsequent occurrences have no effect. Thus, we can skip hash computations for e 's repeated appearances.

To implement this optimization, we introduce an auxiliary data structure to record the set of items observed within the current window. At the end of the window, we iterate through this set and insert each item into the Cold Filter. This mechanism is implemented by the Stage 1 Burst Filter, which effectively mitigates the throughput reduction caused by the Cold Filter.

D. Burst Filter (Stage 1)

Fig. 8 illustrates the structure of the Burst Filter. It consists of w buckets, each containing γ cells, where each cell stores the ID of an item. When an item e arrives, it is hashed into the $f(e)$ -th bucket $B[f(e)]$ using the hash function $f(\cdot)$. The bucket is then traversed, and the following cases are handled:

- 1) If e is already in the bucket, no action is taken.
- 2) If e is not in the bucket and there is an empty cell, e is inserted into the empty cell.
- 3) If e is not in the bucket and there are no empty cells, e is inserted into the Cold Filter in Stage 2.

At the end of each time window, all items in the buckets are iterated over and inserted into the Stage 2 Cold Filter. The Burst Filter is then cleared by emptying all buckets.

• **Example Explanation:** To demonstrate why adding an additional Burst Filter layer can reduce the number of hash computations to increase the throughput, we present an illustrative example. As shown on the left side of Fig. 9, consider only the L_1 layer of the Cold Filter, which uses two hash functions, h_1 and h_2 . Suppose an item e_9 arrives 100 times within the current window. This would require $100 \times 2 = 200$ hash computations.

With the Burst Filter added, as shown on the right side of Fig. 9, we first compute the hash function $f(\cdot)$ 100 times to insert e_9 into the Burst Filter. At the end of the window, when traversing the items in the Burst Filter, e_9 is inserted into the Cold Filter, requiring only 2 additional hash computations (for

Algorithm 1: End_of_Window() $\triangleright$ Flush Stage 1 and reset per-window flags.

Input: Burst Filter bucket array B , Cold Filter arrays L_1, L_2 , Hot Part bucket array L_3

```

1 foreach bucket  $b \in B$  do
2   foreach cell  $c \in b$  do
3     if  $c.id \neq null$  then
4       if // flush Stage 1 items to downstream stages
         Insert_Stage2( $c.id$ ) = false then
5         | Insert_Stage3( $c.id$ );
6       end
7        $c.id \leftarrow null$ ;
8     end
9   end
10 end
11 for  $i \leftarrow 1$  to  $d_1$  do
12   foreach entry  $x \in L_1[i]$  do
13     |  $x.flag \leftarrow on$ ;
14   end
15 end
16 for  $i \leftarrow 1$  to  $d_2$  do
17   foreach entry  $x \in L_2[i]$  do
18     |  $x.flag \leftarrow on$ ;
19   end
20 end
21 foreach bucket  $b \in L_3$  do
22   foreach entry  $x \in b$  do
23     |  $x.flag \leftarrow on$ ;
24   end
25 end

```

h_1 and h_2). This results in a total of 102 hash computations, saving $200 - 102 = 98$ hash computations. The difference becomes even more significant when the Cold Filter employs more hash functions. We formally quantify the reduction in hash computations provided by the Burst Filter in Theorem V.2 in the subsequent mathematical analysis section.

E. Complete Insertion of Hypersistent Sketch

The complete insertion workflow of the Hypersistent Sketch spans its three stages, as described in Fig. 5 and the preceding sections. The process proceeds from Stage 1 to Stage 3 as follows: When an item e arrives, it is first inserted into the Burst Filter. At the end of each time window, all items stored in the Burst Filter are retrieved and sequentially inserted into the Cold Filter. If an item's persistence estimate exceeds the threshold $\Delta_1 + \Delta_2$ in the Cold Filter (indicating an overflow), it is inserted into the Hot Part. The pseudocode for this process is presented in Appendix C, Available online. At the end of each time window, Stage 1 acts as a temporary per-window cache and flushes all valid items sequentially to the downstream stages. After that, Stage 1 is cleared, while Stage 2 and Stage 3 retain their persistence states but reset all per-window on-off flags. This procedure is summarized in Algorithm 1.

F. Persistence Query

After the data stream has ended, the persistence of an item e in the Hypersistent Sketch is queried through the Cold Filter and Hot Part, following these steps:

- 1) *Query L_1 of the Cold Filter:* Hash e into multiple cells of L_1 using the corresponding hash functions, and record the minimum persistence value v_1 among these cells. If $v_1 < \Delta_1$, return v_1 .
- 2) *Otherwise, if $v_1 \geq \Delta_1$, proceed to L_2 :* Hash e into multiple cells of L_2 using the corresponding hash functions, and record the minimum persistence value v_2 among these cells. If $v_2 < \Delta_2$, return $\Delta_1 + v_2$.
- 3) *Otherwise, if $v_2 \geq \Delta_2$, query the Hot Part:* Retrieve the value v_3 of e in the Hot Part (via collision-free ID matching), and return $\Delta_1 + \Delta_2 + v_3$.

This staged approach achieves $O(1)$ average query latency by: (1) filtering 98.7% cold items at L_1 (per our analysis in Section V-B), and (2) It can be expected that if e is a cold item, its persistence is determined within L_1 . If its persistence is slightly larger, it will be handled in L_2 . Only when e is a hot item will it be processed in the Hot Part, (3) The Hot Part resolves hash collisions deterministically for items with $v_2 \geq \Delta_2$ through full ID storage, which stores the full IDs of items. This avoids hash collisions for hot items and supports tasks such as persistent item detection, which require complete item IDs. We support two query modes: **After-window querying** returns fully updated persistence values, reflecting all insertions within the window; **In-window querying** requires an additional hash check for items still being processed in the Burst Filter (Stage 1). Since the vast majority of items are handled at the L_1 layer, only a small fraction of items enter the L_2 layer, and even fewer hot items enter the Hot Part. As a result, both query modes maintain $O(1)$ average query latency. The pseudocode is presented in Algorithm 2.

G. A Running Example

Fig. 10 illustrates a running example of the Hypersistent Sketch. The parameters for each layer are as follows:

- **Burst Filter:** Uses a single hash function to locate a bucket, with each bucket containing 4 cells.
- **Cold Filter:** For L_1 layer, we use 2 hash functions, with a threshold $\Delta_1 = 15$. For L_2 layer, we use 2 hash functions, with a threshold $\Delta_2 = 100$.
- **Hot Part:** Uses a single hash function to locate a bucket, with each bucket containing 3 cells.

Examples of insertion into each layer are shown below.

1) *Inserting Into Burst Filter:* Note that every item will first enter the Burst Filter.

- *Case 1:* Insert e_1 . We find that the corresponding bucket does not contain e_1 but has an empty cell, so e_1 is placed into the bucket.
- *Case 2:* Insert e_2 . We find that e_2 already exists in the corresponding bucket, so no action is taken.
- *Case 3:* Insert e_3 . We find that the corresponding bucket does not contain e_3 , and the bucket is full, so e_3 is inserted into the Cold Filter instead.

2) *Inserting Into the Cold Filter:* Note that items enter the Cold Filter only in two cases: (1) when the Burst Filter's bucket is full within the window and cannot accommodate more items, and (2) after the window ends, when items from the Burst Filter are transferred to the Cold Filter.

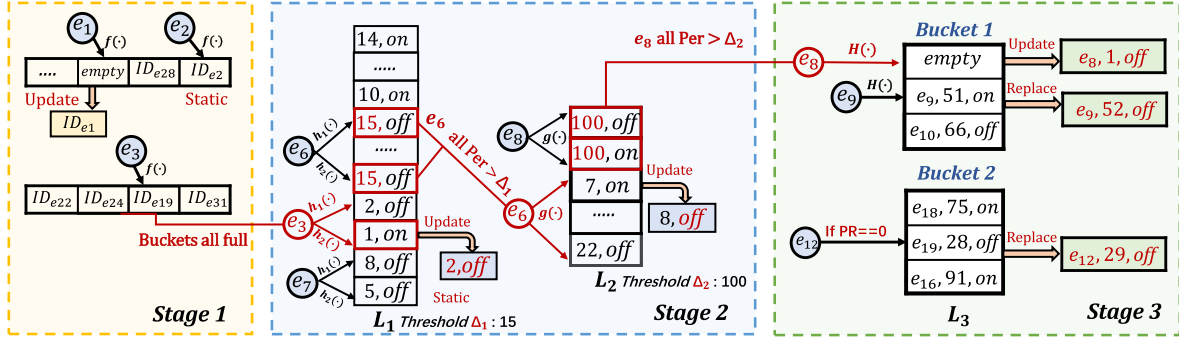


Fig. 10. An example of Hypersistent Sketch.

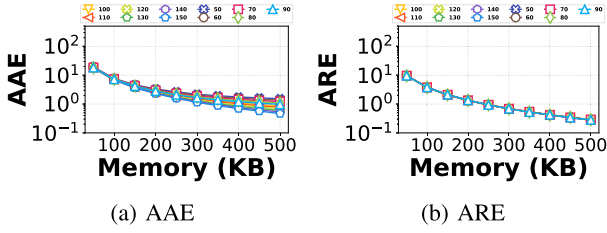
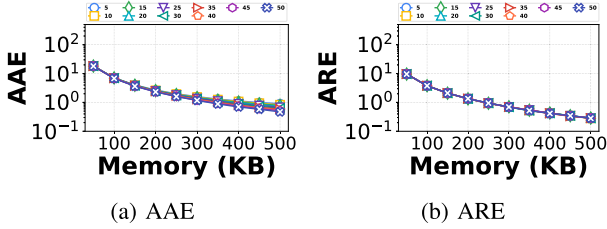
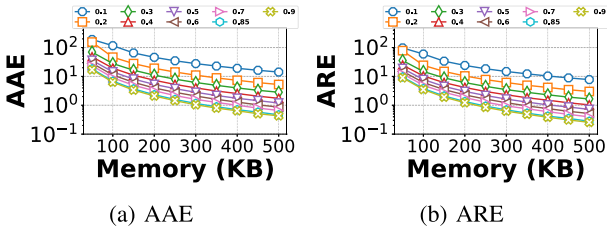
Fig. 11. Impact of Δ_1 on PE.Fig. 12. Impact of Δ_2 on PE.

Fig. 13. Impact of cold & hot memory ratio on PE.

- *Case 4:* e_3 is mapped to two cells in L_1 . The smaller of the two cells is found, and since its flag is “on”, the counter is incremented, and the flag is set to “off”.
- *Case 5:* e_7 is mapped to two cells in L_1 . Since both flags are “off”, no action is taken.
- *Case 6:* e_6 is mapped to two cells in L_1 . Since the minimum value of these cells reaches the threshold $\Delta_1 = 15$, e_6 is moved to L_2 . Similarly, the two cells in L_2 are found, and the smallest is updated.
- *Case 7:* e_8 (assuming Δ_1 is reached in L_1) is mapped to two cells in L_2 . Since the threshold $\Delta_2 = 100$ is met, it suggests that e_8 's persistence may exceed $\Delta_1 + \Delta_2 = 115$. Thus, e_8 is considered a hot item and is inserted into the Hot Part.

Algorithm 2: Query(e).

Input: Item e , Item number $packet_seq$, Window length $window_len$
Output: The persistence of e

```

1   $ret \leftarrow 0$ ;
2  if  $packet\_seq \% window\_len \neq 0$  then
3    for  $cell \in B_f(e)$  do
4      if  $cell.id = e.id$  then
5         $ret \leftarrow 1$ ;  $\triangleright$  find out whether the current
        item exists in the Brust Filter.
6      end
7    endfor
8  end
9   $v1 \leftarrow \min_{1 \leq i \leq d_1} (L_1[i][h_i(e)])$ ;
10 if  $v1 < \Delta_1$  then
11    $ret \leftarrow ret + v1$ ;  $\triangleright L_1$  of Cold Filter
12 end
13  $ret \leftarrow ret + v1$ ;
14  $v2 \leftarrow \min_{1 \leq i \leq d_2} (L_2[i][g_i(e)])$ ;
15 if  $v2 < \Delta_2$  then
16    $ret \leftarrow ret + v2$ ;  $\triangleright L_2$  of Cold Filter
17 end
18  $ret \leftarrow ret + v2$ ;
19  $bucket \leftarrow L_3[H(e)]$ ;
20 for  $entry \in bucket$  do
21   if  $entry.key = e$  then
22      $ret \leftarrow ret + entry.per$ ;  $\triangleright$  Hot Part
23   end
24 endfor
25 return  $ret$ ;

```

3) *Inserting Into the Hot Part:* Note that only items with persistence greater than the Cold Filter's combined threshold $\Delta_1 + \Delta_2$ are inserted into the Hot Part.

- *Case 8:* Insert e_8 . We find that e_8 is not present in the bucket, but there is an empty slot, so e_8 is inserted into the empty slot.
- *Case 9:* Insert e_9 . We find that e_9 already exists in its corresponding bucket, so we update its counter and flag.
- *Case 10:* Insert e_{12} . We find that e_{12} is not in its bucket, and the bucket has no available slots. We locate the item with the smallest persistence in the bucket, e_{19} , and replace it with probability $\frac{1}{28+1}$. If the replacement is successful,

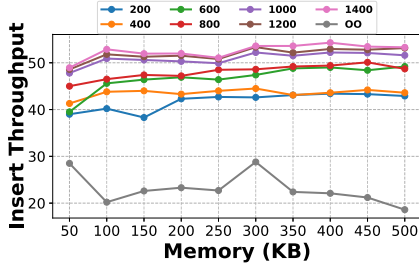


Fig. 14. Insert throughput of Burst Filter of different sizes.

we insert e_{12} and continue updating its counter and flag. If the replacement fails, we return.

H. SIMD Optimization

In this section, we optimize the Burst Filter (Stage 1) for inserting items by exploiting Single Instruction Multiple Data (SIMD) instructions, as most insertions within a window are processed in the Burst Filter. SIMD instructions enable data parallelism through vectorization, which effectively accelerates sequential access operations. Our key optimization targets the item lookup process in the Burst Filter. Traditionally, inserting an item e_i requires checking whether it exists in bucket $B[f(e_i)]$ by sequentially comparing it with each stored entry, which becomes inefficient when multiple items are present. SIMD-based parallel comparisons address this inefficiency. By utilizing these 128-bit SIMD operations, we reduce the number of comparison operations required, accelerating the bucket scanning process by approximately four times compared to traditional sequential scanning. We also demonstrate how SIMD instructions enhance the performance of the Burst Filter. As described in Section IV-D, to insert an item e_i , the Burst Filter first checks whether it is stored in $B[f(e_i)]$. Specifically, this involves sequentially comparing the ID of e_i with the IDs stored in $B[f(e_i)]$. By leveraging SIMD instructions, the Burst Filter can match item IDs in parallel. For example, assume the ID length is 4 bytes, and each bucket stores 4 items. The pseudo-code for the insertion, implemented using AVX2 SIMD instructions, is presented in Algorithm 3.

What's more, the SIMD optimization is not limited to the Burst Filter. It can also be applied to the Hot Part, as long as the underlying base sketch used in the Hot Part itself supports SIMD implementation.

V. MATHEMATICAL ANALYSIS

We illustrate the advantages of Hypersistent Sketch method using mathematical tools in this section. On-Off Sketch Version 2 is used as an example for demonstration and the proofs for other methods follow a similar rationale.

A. Burst Filter: Correctness and Computational Benefit

We next analyze the Burst Filter from two aspects. First, we show that introducing the Burst Filter does not change the final persistence value of any item. Second, we quantify how the Burst Filter reduces redundant downstream hash computations and explain why its throughput benefit becomes larger when the stream is more skewed.

a) *Burst Filter sizing*: Let W denote the number of arrivals in one time window, and let U_t denote the number of distinct items appearing in window t . Clearly, $U_t \leq W$. Therefore, the

Algorithm 3: Scanning a Bucket With SIMD.

Input: Item e . Pointer p is the start of bucket cells

Output: Index of the matched entry or -1 (no found)

```

1: __m128i item = _mm_set1_epi32(e)
2: __m128i * keys = (__m128i *) p
3:   ▷ Step 1: Duplicate item  $e$  into a 128-bit register
4: __m128i a_comp = _mm_cmpeq_epi32(item, keys[0])
5: __m128i b_comp = _mm_cmpeq_epi32(item, keys[1])
6: __m128i c_comp = _mm_cmpeq_epi32(item, keys[2])
7: __m128i d_comp = _mm_cmpeq_epi32(item, keys[3])
8:   ▷ Step 2: Compare item with four bucket cells in parallel
9: a_comp = _mm_packs_epi32(a_comp, b_comp)
10: c_comp = _mm_packs_epi32(c_comp, d_comp)
11: a_comp = _mm_packs_epi32(a_comp, c_comp)
12:   ▷ Step 3: Pack comparison results together
13: matched = _mm_movemask_epi8(a_comp)
14:   ▷ Step 4: Convert SIMD comparison results to mask
15: return matched  $\neq 0$  ? TZCNT(matched) : -1
16:   ▷ Return index of first match or -1 if no match

```

Burst Filter should be sized according to the within-window distinct-item count rather than the raw number of arrivals. This observation is important because, in skewed streams, U_t is usually much smaller than W .

Theorem V.1: Assume that the downstream Cold Filter and Hot Part both follow the per-window on/off update rule. Then adding the Burst Filter does not change the final persistence value of any item.

Proof: Fix an item e and a window t . By definition, the contribution of window t to the persistence of e is either 0 or 1.

If e does not appear in window t , then both the original pipeline and the Burst-Filter-enhanced pipeline contribute 0 to the persistence of e .

Now suppose that e appears at least once in window t . There are two cases.

First, if e is successfully admitted into the Burst Filter, then the first occurrence stores e in Stage 1, and all later occurrences of e in the same window cause no additional downstream update. At the end of the window, e is flushed once from the Burst Filter to the Cold Filter. Hence the downstream structures observe exactly one effective update from window t .

Second, if the corresponding Burst Filter bucket is full, then e is forwarded immediately to Stage 2. In this case, the downstream on/off logic still guarantees that window t contributes at most one effective update for item e .

Therefore, whenever e appears in window t , the enhanced pipeline still contributes exactly one effective downstream update; otherwise it contributes zero. Hence the final persistence value is unchanged by introducing the Burst Filter. $\square$

Theorem V.2: Consider one window with A arrivals. Let U be the number of distinct items successfully recorded by the Burst Filter in this window, and let F be the number of arrivals that cannot be admitted into the Burst Filter because their target buckets are full and are therefore forwarded directly to the Cold Filter. Let H denote the number of items that further overflow from the Cold Filter to the Hot Part. Let c_{CF} be the average number of hash computations required by the Cold Filter for one effective downstream update, and let c_{HP} be the average number of hash computations in the Hot Part. Then the total number of hashes without the Burst Filter:

$$C_{NoBF} = c_{CF}A + c_{HP}H,$$

whereas with the Burst Filter it becomes

$$C_{BF} = A + c_{CF}(U + F) + c_{HP}H.$$

Therefore,

$$\frac{C_{BF}}{C_{NoBF}} = \frac{A + c_{CF}(U + F) + c_{HP}H}{c_{CF}A + c_{HP}H}.$$

In particular, when the stream is skewed and the Burst Filter is sized so that $U + F \ll A$, the Burst Filter yields a substantial reduction in the total number of hash computations.

Proof: Without the Burst Filter, every arrival reaches the Cold Filter, so the Cold Filter cost is proportional to A . Items that further overflow from the Cold Filter to the Hot Part additionally incur $c_{HP}H$ hash computations. Hence,

$$C_{NoBF} = c_{CF}A + c_{HP}H.$$

With the Burst Filter, each arrival first incurs one Stage 1 hash computation, contributing A in total. Among these arrivals, only the U distinct items successfully stored in the Burst Filter are flushed once at the window boundary. In addition, F arrivals cannot be admitted because their target Burst Filter buckets are full and are therefore forwarded directly to the Cold Filter. Thus the Cold Filter cost is proportional to $U + F$. The same downstream hot-item overflows contribute $c_{HP}H$ in the Hot Part. Summing these terms yields

$$C_{BF} = A + c_{CF}(U + F) + c_{HP}H.$$

The ratio formula follows immediately. When the stream is skewed, many repeated arrivals correspond to the same small set of distinct items within a window, so U is much smaller than A . If the Burst Filter is provisioned according to the within-window distinct count, then F is also small. Hence $C_{BF} \ll C_{NoBF}$, which explains the throughput gain brought by the Burst Filter. $\square$

b) Distribution-dependent interpretation: Let p_i denote the probability of item i within a window containing A arrivals. Then the expected number of distinct items observed in the window is

$$\mathbb{E}[U] = \sum_i (1 - (1 - p_i)^A).$$

This immediately yields the following special cases.

For a uniform distribution over K items,

$$\mathbb{E}[U] = K \left(1 - \left(1 - \frac{1}{K} \right)^A \right).$$

For a Zipf distribution with exponent $s > 1$ over N items,

$$p_k = \frac{k^{-s}}{H_{N,s}}, \quad H_{N,s} = \sum_{j=1}^N \frac{1}{j^s},$$

and thus

$$\mathbb{E}[U] = \sum_{k=1}^N \left(1 - \left(1 - \frac{k^{-s}}{H_{N,s}} \right)^A \right).$$

Exponential-like popularity models are handled in the same way by substituting the corresponding item probabilities $\{p_i\}$ into the general formula above. Therefore, the heavier the skew, the smaller the ratio $\mathbb{E}[U]/A$, and the larger the computational benefit of the Burst Filter. Under common settings where the Cold Filter dominates the downstream hashing cost and the Burst

Filter absorbs most repeated arrivals within a window, the overall throughput gain can approach about $2\times$.

B. Persistence Estimation

1) *Error Bound: Theorem V.3:* Let $\hat{p}_i$ be the estimated persistence of our method. We have

$$p_i \leq \hat{p}_i \leq T. \quad (1)$$

Proof: Let the threshold for L_1 be Δ_1 and the threshold for L_2 be Δ_2 . The number of time windows in S is T . If $\hat{p}_i > \Delta_1$, the item is transmitted to L_2 ; if $\hat{p}_i > \Delta_1 + \Delta_2$, it is sent to L_3 . The number of hash functions in L_1 , L_2 , and L_3 is d_1 , d_2 , and d_3 , respectively. Thus, $\hat{p}_i$ can be expressed as:

$$\hat{p}_i = \hat{p}_i^1 + \hat{p}_i^2 + \hat{p}_i^3,$$

where $\hat{p}_i^1$, $\hat{p}_i^2$, and $\hat{p}_i^3$ represent the estimated persistence in L_1 , L_2 , and L_3 . Subsequently, we determine the upper and lower boundaries of $\hat{p}_i$. For each $\hat{p}_i^j$ ($j = 1, 2, 3$), each arriving item can trigger the mapping, causing $\hat{p}_i^j$ to increase. Consequently, we have $\hat{p}_i^j \geq p_i^j$ ($j = 1, 2, 3$). In each time window, $\hat{p}_i^j$ can increase by at most 1, which results in $\hat{p}_i \leq T$. $\square$

In summary, we have $p_i \leq \hat{p}_i \leq T$. This result indicates that the estimated persistence provided is within a reasonable range and will not introduce significant deviation when utilized.

Theorem V.4: Let n , m and L be the number of buckets in layer L_1 , L_2 and L_3 , $l = \frac{e}{e}$ and $d = \ln(\frac{1}{\delta})$. For small data streams, we obtain

$$\mathbb{P}(\hat{p}_i \leq p_i + \epsilon \|p\|_1) \geq 1 - \delta. \quad (2)$$

For medium data streams, where $\epsilon = \frac{e}{n \times m}$ and $\delta = e^{-d_1 - d_2}$, it follows that

$$\mathbb{P}(\hat{p}_i \leq p_i + \epsilon \|p\|_1 \times \|p\|_1^1) \geq 1 - \delta. \quad (3)$$

For large data streams, $\epsilon_1 = \frac{e}{n \times m \times L}$ and $\delta = e^{-d_1 - d_2 - d_3}$, we conclude that

$$\mathbb{P}(\hat{p}_i \leq p_i + \epsilon_1 \|p\|_1 \times \|p\|_1^1 \times \|p\|_1^2) \geq 1 - \delta. \quad (4)$$

Proof: Please refer to Appendix B.1. $\square$

2) *Comparison With Related Work Without Cold Filter:* *Theorem V.5:* Let $\hat{p}_i^{OO}$ be the estimated persistence of the On-Off Sketch. Under the same memory conditions, we filter the entire data stream by assigning a different number of counters and d_i for small, medium, and large data streams. For simplicity, we use the same names for the hash functions in the On-Off Sketch as in our method, even though they have different numbers of counters. Let $\Delta_j^{OO} p_i = C_j[h_j(e_i)] - p_i$, where $p_i = \min_{1 \leq j \leq d} (C_j[h_j(e_i)])$, and d represents the number of hash functions in the On-Off Sketch. There is:

$$E(\Delta_j^{OO} p_i) > E(\Delta_j p_i).$$

Proof: For small data streams, $E[\Delta_j p_i]$ can be represented by the following formula in our method and the On-Off Sketch:

$$E[\Delta_j p_i] = \sum_{t \in \hat{P}_i} \left[1 - \left(1 - \frac{1}{n} \right)^{|E_t|} \right],$$

Since we utilize low-byte storage, there is more counters, allowing us to conclude that $E(\Delta_j p_i) \leq E(\Delta_j^{OO} p_i)$. For medium

data streams, we have

$$1 - \left(1 - \frac{1}{m}\right)^{|E_t|} < 1.$$

Therefore, it follows that

$$E(\Delta_j^{OO} p_i) > 1 - \left(1 - \frac{1}{n}\right)^{|E_t|} > E(\Delta_j p_i).$$

The above inequalities indicate that Hypersistent Sketch method is superior to the On-Off Sketch. The Cold Filter improved the effectiveness of data processing. $\square$

C. Finding Persistent Items

Theorem V.6: Let $\hat{p}_i = B[h_1(e_i)][e_i]$ and denote the On-Off Sketch estimate as $\hat{p}_i^{OO}$, where $B[i]$ is the i th bucket. We have the following inequality:

$$p_i \leq \hat{p}_i \leq \hat{p}_i^{OO} \leq T. \quad (5)$$

Proof: Please refer to Appendix B.2, Available online. $\square$

Therefore, Hypersistent Sketch outperforms the On-Off Sketch without Cold Filter in finding persistent items.

D. Skewness-Aware and Threshold Sensitivity Analysis

The detailed discussion of analyses for skewness-aware and threshold sensitivity are provided in the appendix, Available online [52], [54]. We present only the relevant theorems and conclusions here.

Theorem V.7: Assume the persistence follows a Zipf distribution with parameter s , i.e., the persistence of the i -th most frequent item is:

$$p_i = \frac{1}{i^s H_N^{(s)}}, \quad \text{where } H_N^{(s)} = \sum_{k=1}^N \frac{1}{k^s}.$$

The expected error upper bound of the Hypersistent Sketch satisfies:

$$\mathbb{E}[\hat{p}_i - p_i] \leq \frac{H_N^{(s)}}{n} + \frac{H_N^{(s-1)}}{m},$$

where n and m are the number of counters in L_1 and L_2 layers of the Cold Filter, respectively.

This indicates that our method is adaptable to data with different skews. **When the skewness is large, our method performs better; when the skewness is not large, the data distribution is relatively uniform, and our method is not inferior to the comparison method.**

Theorem V.8: Let the Cold Filter thresholds be parameterized as:

$$\Delta_1 = k_1 \cdot \frac{\log n}{\log \log n}, \quad \Delta_2 = k_2 \cdot \Delta_1 = k_1 k_2 \cdot \frac{\log n}{\log \log n}.$$

where k_1, k_2 are tunable constants. The memory-error trade-off satisfies:

$$\text{Memory Efficiency} \propto \frac{1}{k_1 k_2}, \quad \text{Relative Error} \propto \frac{\sqrt{k_1}}{n^{1/2}} + \frac{\sqrt[3]{k_2}}{m^{1/3}}.$$

Pareto optimality is achieved when:

$$k_1 = \Theta\left(\sqrt{\frac{n}{\log n}}\right), \quad k_2 = \Theta\left(\sqrt[3]{\frac{m}{\log m}}\right).$$

This ratio automatically adapts to data scale while maintaining near-optimal performance. The theorem shows that **our method behaves differently for different thresholds and gives the memory allocation and the relationship between error and threshold parameters. In addition, the Pareto optimality of the theoretical threshold is given at the end, which can guide us in setting the corresponding threshold parameters in the experiment.**

E. Time and Space Complexity Analysis

The overall insertion and query time complexity of Hypersistent Sketch is determined by the operations performed in each stage. **Insertion complexities of Burst Filter, Cold Filter (L1, L2) and Hot Part are $O(1)$, $O(d_1)$ and $O(1)$ respectively. Query complexities of Burst Filter, Cold Filter (L1, L2) and Hot Part are $O(\gamma)$, $O(d_1 + d_2)$ and $O(1)$.**

The space complexity of Hypersistent Sketch is determined by memory allocation across the three stages. **Space complexities of Burst Filter, Cold Filter (L1, L2) and Hot Part are $O(w\gamma)$, $O(d_1 w + d_2 w)$ and $O(\lambda\beta)$ respectively.**

Overall, the time complexity of the Burst Filter, Cold Filter, and Hot Filter in our method is $O(d_1 + d_2) + O(\gamma)$. Hypersistent Sketch divides the available memory into three parts (M_1 for Burst Filter, M_2 for Cold Filter, M_3 for Hot Filter), so the total space complexity is:

$$O(M_1 + M_2 + M_3) \quad (\text{which can be combined into } O(M)).$$

In stream processing applications, M is usually a fixed amount set in advance based on accuracy requirements or hardware constraints, and it does not grow linearly with the data stream size N . Therefore, it is often represented as $O(M)$.

VI. EXPERIMENTAL RESULTS

This section presents experimental results, including experimental setup (VI-A), performance on persistence estimation (VI-C), and finding persistent items (VI-D). Finally, we explore throughput enhancement using SIMD instructions (VI-E).

A. Experimental Setup

1) *Implementation:* Experiments were conducted on a six-core Intel i5-8400 (2.80 GHz, 16 GB DRAM) with 32 KB L1, 256 KB L2, and 9 MB shared L3 cache, ensuring sufficient computational resources.

2) *Metrics:* *Insert Throughput:* N/T million operations per second (Mops), where N is the total number of insertions and T is the total time taken. Median values are reported over five experimental runs.

Query Throughput: Q/T million queries per second (Mqps), where Q is the total number of queries and T is the total time taken. Median values are reported over five experimental runs.

AAE (Average Absolute Error): $\frac{1}{|\Phi|} \sum_{e_i \in \Phi} |P(e_i) - \hat{P}(e_i)|$, where $P(e_i)$ is the real persistence of item e_i , $\hat{P}(e_i)$ is the estimated persistence, and Φ is the query set.

ARE (Average Relative Error): $\frac{1}{|\Phi|} \sum_{e_i \in \Phi} \frac{|P(e_i) - \hat{P}(e_i)|}{P(e_i)}$, evaluating the error rate of estimated persistence.

F1-Score: The F1-Score is a balanced metric for detecting persistent items, combining precision and recall to minimize both false positives and false negatives. $\text{F1-Score} = \frac{2 \cdot \text{TP}}{2 \cdot \text{TP} + \text{FP} + \text{FN}}$,

where TP (True Positives) are correctly identified persistent items, FP (False Positives) are non-persistent items misclassified as persistent, and FN (False Negatives) are persistent items missed by the model.

FNR (False Negative Rate): The FNR quantifies the proportion of persistent items that are not identified: $FNR = \frac{FN}{FN+TP}$, where FN (False Negatives) refers to persistent items that are not reported, and TP (True Positives) represents persistent items correctly identified.

FPR (False Positive Rate): The FPR measures the proportion of non-persistent items that are incorrectly identified as persistent: $FPR = \frac{FP}{FP+TN}$, where FP (False Positives) refers to non-persistent items incorrectly identified as persistent, and TN (True Negatives) represents non-persistent items correctly identified.

3) **Dataset:** Unless otherwise specified, we use the term *item* consistently throughout the paper.

CAIDA Dataset: We use the CAIDA Equinix-Chicago trace, where the 4-byte source IP address is used as the item ID. The 5-second trace used in the main experiments contains 2.49 M packets and 109,534 distinct items. For large-scale evaluation, we additionally use a 2-minute trace containing 53.72 M packets and 1.71 M distinct items. Most items have persistence below 50.

Big CAIDA Dataset: We use large-scale anonymous CAIDA traces containing 30 M packets and 543,996 distinct items. The 4-byte source IP address is the item ID.

MAWI Dataset: We use a MAWI trace [55], where the item ID is a 13-byte five-tuple-based key. The evaluated trace contains 2 M packets and 200,471 distinct items, with 266 hits and 125 items exceeding the persistence threshold.

Campus Dataset: We use campus gateway traces, where the item ID is also a 13-byte five-tuple-based key. The evaluated subset contains 10 M packets and 259,948 distinct items, with 1,333 hits and 677 items exceeding the persistence threshold.

Synthetic Datasets: The synthetic datasets are generated by the Web Polygraph tool and follow Zipf distributions with skewness from 1.5 to 2.5. We use 4-byte generated keys as item IDs. Each dataset contains about 9.8 M packets.

4) **Methodology:** Hypersistent Sketch (HS) was implemented in C++ using BOBHash, with distinct random seeds for different hash functions, and the seeds were regenerated at each time window to preserve probabilistic randomness. Since HS is a *general enhancement framework* rather than a standalone sketch algorithm, we evaluate each baseline sketch together with its HS-enhanced counterpart under the same experimental settings. For *persistence estimation*, we consider Count-Min Sketch (CM) [46], On-Off Sketch (OO) [56], and WavingSketch (WS) [48], and evaluate both the original sketches and their HS-enhanced versions, i.e., H-CM, H-OO, and H-WS, using AAE and ARE. Tight Sketch (TS) [50], P-Sketch (PS) [49], and Stable Sketch [51] are not included in this task because they do not maintain whole-item persistence records in the sketch, making them unsuitable for estimating the persistence of complete items. Evaluations were conducted under varying memory sizes (50–500 KB) with a persistence window of 3000. For CM and WS, half of the memory was allocated to the Bloom Filter [47] to reduce false positives, same as the configuration in On-Off Sketch [34]. OO used the original three-layer structure. In HS, the Burst Filter size was set to window size/100 KB. After reserving the Burst Filter memory, the remaining memory was

divided between the Cold Filter and the Hot Part using a unified ratio of 17:3.

For *finding persistent items*, we consider Small-Space (SS) [57], WavingSketch (WS) [48], On-Off Sketch (OO) [56], Tight Sketch (TS) [50], and P-Sketch (PS) [49], and similarly evaluate their HS-enhanced versions using F1-Score, ARE, FNR, and FPR. Evaluations were performed with a persistence window of 1500 and memory ranging from 10 KB to 50 KB. CM and WS allocated half of the memory to the Bloom Filter to ensure a low FPR, while OO, PS, and TS used the original three-layer structures. In HS, the Burst Filter was set to 1 KB, and after reserving the Burst Filter memory, the remaining memory was divided between the Cold Filter and the Hot Part using the same 17:3 ratio.

In addition to the default experimental settings above, we further conduct a parameter sensitivity study on the key hyperparameters of the framework, including the threshold parameters Δ_1 and Δ_2 , the Burst Filter memory size, and the memory partition between the Cold Filter and the Hot Part. Unless otherwise specified, we use $\Delta_1 = 50$ and $\Delta_2 = 150$ for persistence estimation, and $\Delta_1 = 50$ and $\Delta_2 = 120$ for finding persistent items. The detailed sensitivity results are presented in Sections VI-B.

We use the open-source implementations of the compared methods in our experiments. The source code of Hypersistent Sketch can be downloaded from the Website [54], as well as GitHub [52].

B. Sensitivity Analysis

We further study the sensitivity of the *persistence estimation* task to the threshold parameters Δ_1 and Δ_2 . As shown in Figs. 11 and 12, we report both **AAE** and **ARE** under different threshold settings. The results show that the overall estimation accuracy is relatively stable across a broad range of threshold values, indicating that the framework is not overly sensitive to the exact parameter choice. We also observe that when both Δ_1 and Δ_2 are set to relatively larger values, the **AAE** becomes slightly better. This is reasonable because AAE is more influenced by the estimation quality of hot items, and larger threshold values provide stronger protection for such items before they are affected by competition from colder ones. As a result, the absolute estimation error of hot persistent items is slightly reduced.

We then examine the memory partition between the **Cold Filter** and the **Hot Part**. As shown in Fig. 13, we vary the ratio $\frac{\text{Cold Filter}}{\text{Cold Filter} + \text{Hot Part}}$ on the CAIDA dataset and report **AAE** and **ARE**. The results show that a larger Cold Filter ratio generally leads to better performance on both metrics. This is consistent with the nature of persistence estimation, which needs to control the error over *all* items, including a large number of cold items and only a small number of hot items. Since the Cold Filter is mainly responsible for absorbing and maintaining cold items before they interfere with the Hot Part, allocating a larger share of memory to the Cold Filter naturally improves the overall estimation quality. Therefore, for persistence estimation, a larger Cold Filter share is preferable after reserving a small Burst Filter budget.

In addition to the above accuracy-oriented sensitivity study, we further analyze how the memory size of the **Burst Filter** affects **insertion throughput**. As shown in Fig. 14, we vary the

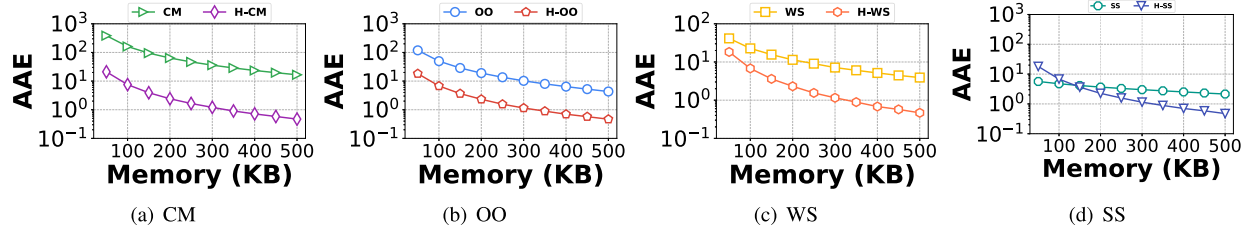


Fig. 15. AAE on persistence estimation vs. memory (CAIDA).

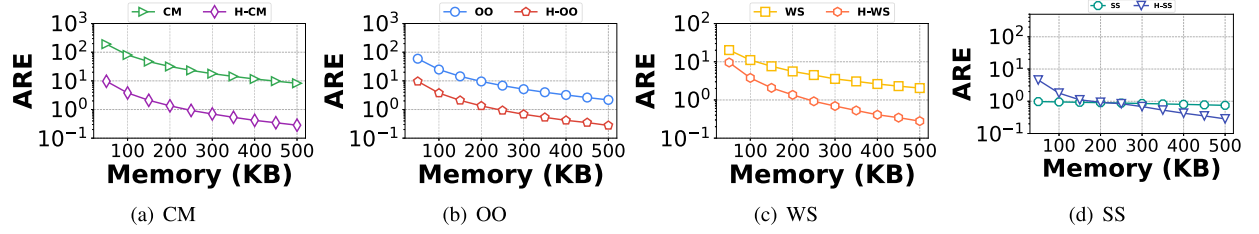


Fig. 16. ARE on persistence estimation vs. memory (CAIDA).

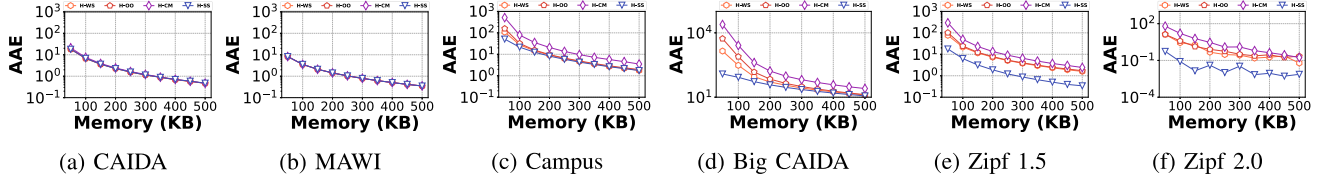


Fig. 17. AAE of persistence estimation on different datasets.

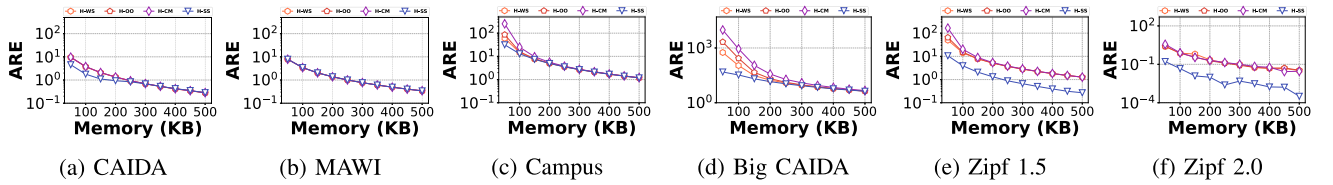


Fig. 18. ARE of persistence estimation on different datasets.

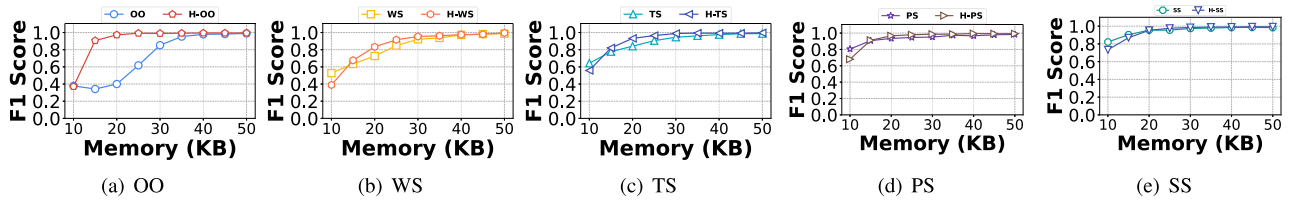


Fig. 19. F1 Score on finding persistent items (CAIDA).

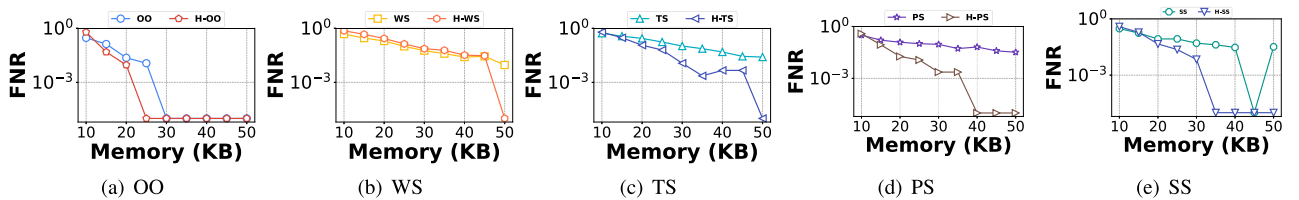


Fig. 20. FNR on finding persistent items (CAIDA).

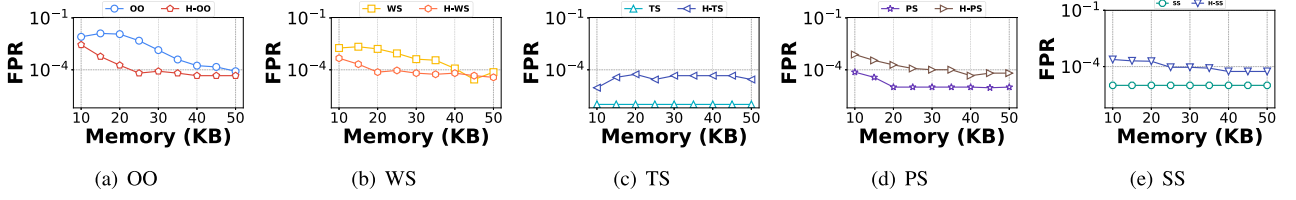


Fig. 21. FPR on finding persistent items (CAIDA).

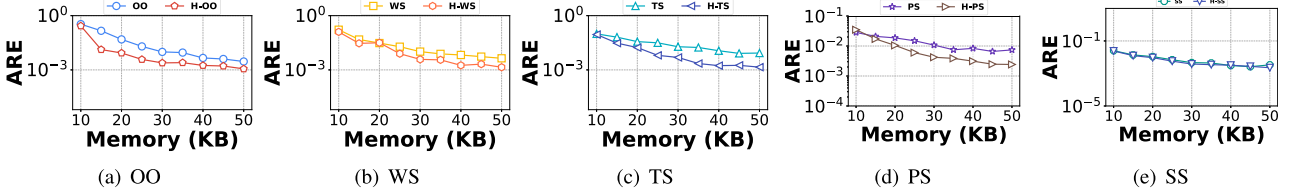


Fig. 22. ARE on finding persistent items (CAIDA).

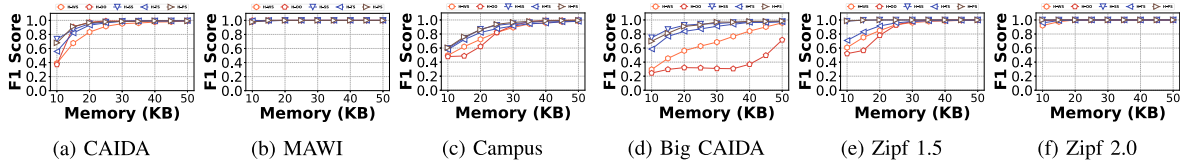


Fig. 23. All sketches' F1-score of finding persistent items on different datasets.

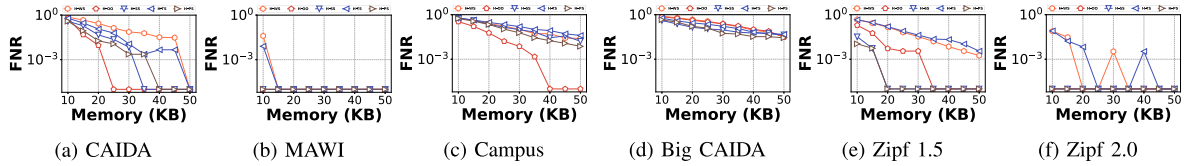


Fig. 24. All sketches' FNR of finding persistent items on different datasets.

Burst Filter memory under different total memory budgets and measure the corresponding insertion throughput.

The Burst Filter is designed purely for **throughput optimization**; it is not an accuracy-enhancing component by itself. Its role is to act as a per-window cache for the downstream Cold Filter and Hot Part: once an item has already appeared in the current window, repeated arrivals of that item no longer need to trigger repeated accesses to the later stages. Therefore, its size should be determined according to the *number of distinct items within one window*, rather than the total number of arrivals in that window.

The results in Fig. 14 show a clear trend. As the Burst Filter becomes larger, insertion throughput improves because more repeated items are absorbed in Stage 1 and fewer redundant accesses are issued to the downstream Cold Filter. However, once the Burst Filter is large enough to cover almost all distinct items within a window, the throughput improvement becomes marginal, which is exactly the expected behavior of a cache-like component. We also note that the reported total memory budget in our experiments *includes* the Burst Filter memory. Therefore, the Burst Filter should occupy only a small fraction of the total memory, just enough to cover the within-window distinct-item count and capture most of the throughput benefit in practice.

C. Persistence Estimation

1) *AAE*: Fig. 15 present the results for Average Absolute Error (AAE) on CAIDA when applying our Hypersistent Sketch framework to four baseline sketches: the Count-Min Sketch (CM), the On-Off Sketch (OO), the Waving Sketch (WS), and the Stable Sketch (SS). In each subfigure, algorithms prefixed with 'H-' (i.e., H-CM) denote the enhanced versions integrated with our framework.

The primary observation is that the Hypersistent Sketch framework consistently improves accuracy. Across nearly all datasets and memory allocations, the enhanced 'H-' variants exhibit a lower AAE than their original counterparts. This improvement is particularly dramatic in certain cases, such as on the CAIDA dataset, where H-CM reduces the AAE by approximately 1.5 orders of magnitude compared to the original CM Sketch. A notable exception occurs on the large-scale Big CAIDA dataset under extremely tight memory constraints (<100 KB), where the enhanced algorithms occasionally underperform the originals. This behavior is attributable to the extreme memory pressure. For a massive dataset like Big CAIDA, a memory allocation of under 100 KB is already insufficient for the baseline sketch. When our framework partitions a portion of this already-scarce memory for the Cold Filter, the benefit gained

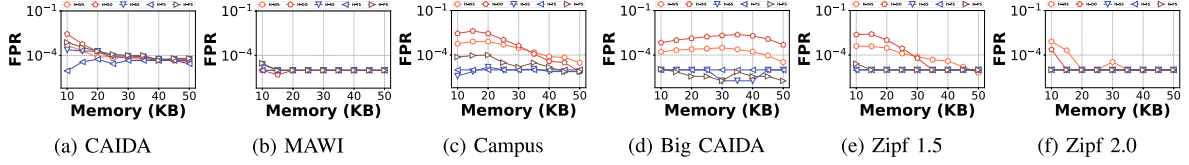


Fig. 25. All sketches' FPR of finding persistent items on different datasets.

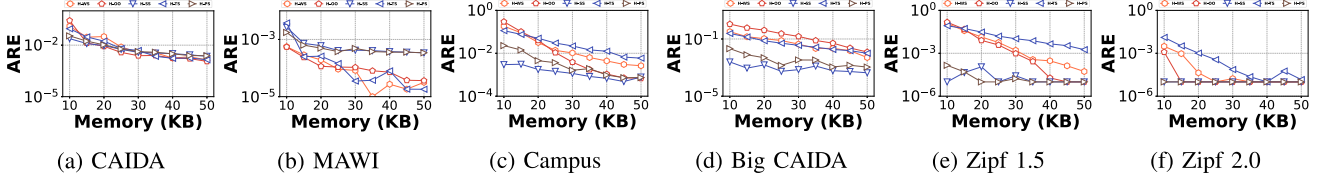


Fig. 26. All sketches' ARE of finding persistent items on different datasets.

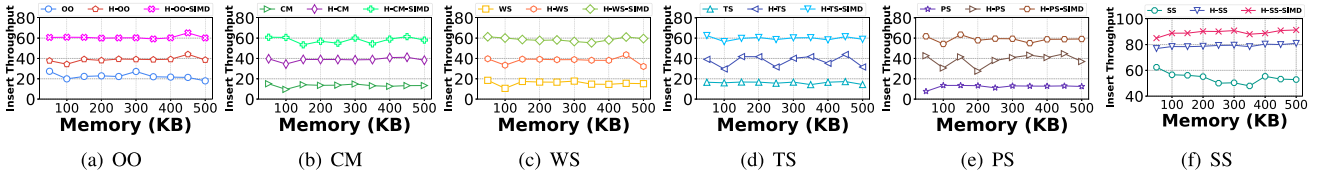


Fig. 27. Insert throughput with/without framework or SIMD in CAIDA.

from filtering cold items is outweighed by the sharp increase in error from the now even more constrained base sketch. However, this small accuracy deficit reveals an important trade-off. As we will demonstrate in the throughput evaluation (Section VI-E), the insertion speed of our enhanced sketches remains far superior to the originals even in these low-memory scenarios. This positions the Hypersistent Sketch framework as a versatile tool: in memory-starved environments, it allows for trading a marginal loss in accuracy for a substantial gain in processing speed. Conversely, when memory is not impractically constrained, our framework provides a comprehensive improvement, outperforming the baseline sketches in both accuracy and speed.

2) *ARE*: Fig. 16 shows the results for Average Relative Error (ARE) on CAIDA. The trends observed are nearly identical to those for AAE presented in the previous section; our framework consistently enhances the performance of the baseline sketches (CM, OO, WS, and SS). Therefore, to avoid redundancy, we will not repeat a detailed analysis.

It is important, however, to consider the distinct characteristics of these two error metrics. AAE is predominantly influenced by the large errors associated with high-persistence (hot) items, while ARE gives more weight to the errors on low-persistence (cold) items. The fact that our Hypersistent Sketch framework demonstrates strong performance improvements on *both* AAE and ARE metrics is significant. It provides compelling evidence of the framework's comprehensive effectiveness, showcasing its ability to reduce estimation errors for both hot and cold items alike.

Integrated Comparison Across All HS-Enhanced Sketches: To identify the best *enhanced* back-end sketch under identical settings, we further present an integrated comparison of all applicable HS-enhanced sketches for *persistence estimation* in Figs. 17 and 18. Different from the pairwise comparisons above, these figures focus only on the HS-enhanced variants, i.e., the

sketches prefixed with “H-”, so that the comparison directly answers which Stage 3 candidate performs best after being integrated into the Hypersistent Sketch framework.

The consolidated results lead to a clear conclusion. Among all HS-enhanced candidates, **H-SS** consistently achieves the lowest AAE and ARE across different datasets and memory settings, showing the best overall accuracy-memory trade-off for *persistence estimation*. This result indicates that, when the same Hypersistent front-end is applied, Stable Sketch serves as the strongest Stage 3 back-end among the evaluated candidates. Therefore, for the *persistence estimation* task, we recommend **H-SS** as the preferred choice.

D. Finding Persistent Items

1) *F1-Score*: We now evaluate the performance on the task of finding persistent items, using the F1-score as the primary metric. Fig. 19 illustrate the F1-scores of four baseline sketches—the On-Off Sketch (OO), Waving Sketch (WS), Tight Sketch (TS), P-Sketch (PS), and Stable Sketch (SS)—both with and without our Hypersistent Sketch framework. The results clearly show that applying our framework yields a higher F1-score for the enhanced ‘H-’ variants across nearly all datasets and memory configurations. For instance, in the case of P-Sketch, while the baseline algorithm has an advantage in low-memory regimes, the enhanced H-PS surpasses it as more memory becomes available. This observation reinforces the conclusion drawn in the previous section: our framework’s ability to enhance performance is most pronounced when the memory allocation is not impractically constrained.

2) *FNR*: Fig. 20 show the False Negative Rate (FNR) for the task of finding persistent items. Similar to our previous findings, the results demonstrate that applying the Hypersistent Sketch framework consistently reduces the FNR compared to the

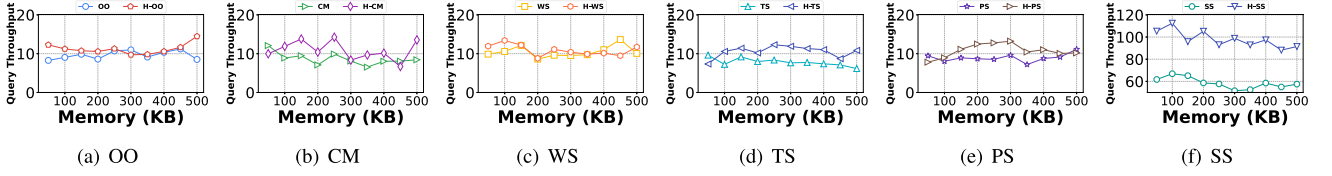


Fig. 28. Query throughput with/without framework or SIMD in CAIDA.

baseline versions across nearly all datasets, memory allocations, and algorithms.

3) *FPR*: Fig. 21 show the False Positive Rate (FPR) for the task of finding persistent items. The conclusions are consistent with those for the FNR, demonstrating that our framework also provides a robust optimization effect across all tested algorithms. To avoid redundancy, we do not repeat the analysis here.

4) *ARE*: The Average Relative Error (ARE) for this task is presented in Fig. 22. The results are comprehensive and conclusive. In every tested scenario, encompassing all baseline algorithms (OO, WS, TS, PS, and SS), datasets, and memory allocations, the enhanced ‘H-’ variants consistently outperform their original counterparts by achieving a lower ARE. This demonstrates the universal effectiveness of our framework in improving the accuracy of identifying persistent items.

Integrated Comparison Across All HS-Enhanced Sketches: To complement the pairwise comparisons above, we further provide a consolidated comparison of all HS-enhanced sketches for the *finding persistent items* task in Figs. 23–26. Specifically, we compare the sketches prefixed with “H-” under the same experimental conditions and report four metrics, namely F1-score, FNR, FPR, and ARE, so as to directly determine which enhanced back-end sketch performs best within the Hypersistent Sketch framework.

The integrated results again show that **H-SS** achieves the best overall performance among all HS-enhanced candidates. In particular, H-SS consistently provides the most favorable balance across F1-score, FNR, FPR, and ARE on different datasets and memory settings, indicating stronger detection quality and robustness than the other enhanced variants. Therefore, for the *finding persistent items* task, we also recommend **H-SS** as the preferred choice. Combined with the persistence estimation results above, these experiments demonstrate that **H-SS** is the best overall Stage 3 candidate among those evaluated in this paper.

E. Throughput and SIMD Acceleration

While the preceding subsections have established that the Hypersistent Sketch framework significantly enhances the accuracy of existing sketches, an equally important question remains: what is its impact on speed? This section addresses this by providing a detailed comparison of insertion and query throughput. We evaluate three distinct versions for each baseline sketch: the original algorithm, the algorithm enhanced with our framework, and the framework-enhanced version with further SIMD optimizations.

SIMD [58] acceleration significantly enhances the performance of HS. Without SIMD, inserting an item into a bucket with 16 cells requires up to 16 comparisons. With SIMD, cells are grouped into four sets of four, and comparisons are performed in parallel using a 128-bit vector, reducing comparisons to just four per insertion.

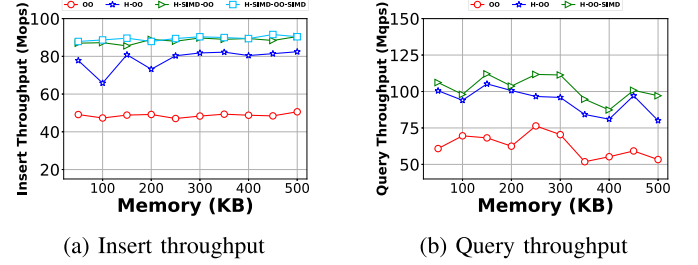


Fig. 29. Insert and query throughput of On-off sketch (OO).

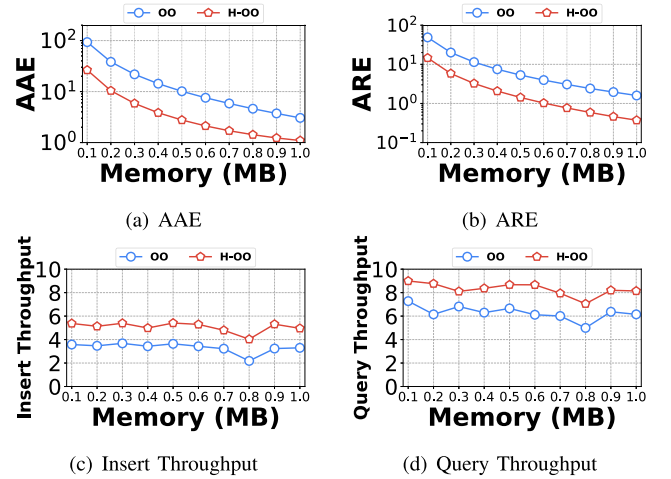


Fig. 30. Performance in Redis implementation.

1) *Insertion Throughput*: Fig. 27 present the insertion throughput results for all five baseline sketches (OO, CM, WS, TS, PS, and SS). The results are striking. On average, applying the Hypersistent Sketch framework alone more than doubles the insertion throughput of the original algorithms. With the addition of SIMD optimizations, the throughput can reach up to three times that of the baseline. Notably, several enhanced algorithms achieve speeds exceeding 100 Mpps (Million packets per second). These findings highlight the exceptional acceleration effect of our Burst Filter and confirm the significant benefit provided by SIMD optimization.

2) *Query Throughput*: The query throughput results are presented in Fig. 28. Two key observations are noteworthy. First, applying the Hypersistent Sketch framework does not degrade query performance; the throughput of the enhanced ‘H-’ variants is nearly identical to that of their original counterparts. This is because the majority of items in a data stream are ‘cold’. Consequently, most queries are resolved by accessing only the first layer (L1) of the Cold Filter. Only the small fraction of ‘hot’ items necessitate traversing deeper to the second layer (L2) and

the main Hot Part, which keeps the average query path short and efficient.

Second, it should be noted that we do not present results for the SIMD-optimized versions in these figures. This is intentional, as our SIMD optimization is implemented exclusively within the Burst Filter to accelerate the insertion process. Since the query operation does not involve the Burst Filter, there is no corresponding impact on query throughput.

3) *SIMD in the Hot Part*: In addition to the Burst Filter, SIMD can also be applied to the *Hot Part*, provided that the underlying base sketch used in the Hot Part itself supports SIMD implementation. To validate this point, we further conduct an experiment on the CAIDA dataset using **On-Off Sketch (OO)**, whose operations can be SIMD-accelerated, and report the results in Fig. 29.

For *insertion throughput*, as shown in Fig. 29(a), enabling SIMD in the Hot Part brings only a marginal additional gain: **H-SIMD-OO-SIMD** is only slightly faster than **H-SIMD-OO**. This is expected because, during insertion, most arriving items are cold items and do not enter the Hot Part frequently. Therefore, the major throughput improvement still comes from applying SIMD to the Burst Filter, while SIMD in the Hot Part contributes only limited additional benefit.

For *query throughput*, however, the Burst Filter does not participate in the query procedure, and the relevant comparison is between **H-OO** and **H-OO-SIMD**, where the latter means that SIMD is applied to the Hot Part. As shown in Fig. 29(b), **H-OO-SIMD** is moderately faster than **H-OO**, with an improvement of about 5% ~ 10%. This indicates that, although SIMD in the Hot Part does not significantly improve insertion throughput, it is still useful for accelerating query/lookup operations.

These results show that SIMD optimizations in the Burst Filter and in the Hot Part are orthogonal. The former mainly improves insertion throughput, whereas the latter mainly benefits query throughput and provides only minor additional acceleration during insertion.

In summary, the framework-enhanced algorithms maintain query speeds that are on par with the original sketches, confirming that our improvements to accuracy and insertion throughput come with no penalty to query performance.

F. Redis Implementation

To validate the practical performance of our Hypersistent Sketch (HS) framework in a real-world software environment, we integrated it into Redis [59], a widely-used in-memory data store. A successful integration not only demonstrates our framework's feasibility beyond simulations but also lays the groundwork for potential applications, such as identifying persistent keys for optimized data tiering or backup strategies in large-scale Redis deployments.

Challenges: Implementing complex, custom data structures as a Redis module presents significant engineering challenges. 1) Redis modules are written in ANSI C, precluding the use of C++ standard libraries and their rich features. 2) All memory must be managed via the Redis Module API, meaning standard functions like `malloc` and data structures such as `std::map` or `std::set` cannot be used directly. 3) The module API, while powerful, is intricate and requires careful handling to ensure the stability and performance of the core database.

Navigating these constraints, we successfully implemented our framework and the On-Off Sketch (OO) baseline as a

RedisBloom [60] module. The module was loaded into Redis server version 6.2.6, running on the same hardware platform as previous experiments. The evaluation was performed using the CAIDA dataset, and the memory allocated to the sketches was varied from 0.1 MB to 1.0 MB.

Fig. 30 presents the results. The accuracy, measured by ARE and AAE, is shown in Fig. 30(a) and (b), respectively. These error rates are consistent with the results from our earlier CPU-based simulations, confirming the correctness of the implementation. More importantly, we evaluate performance by measuring the insertion and query throughput in Millions of Operations Per Second (MOPS). As shown in Fig. 30(c) and (d), the proposed HS consistently outperforms the baseline. Averaged across the memory configurations, HS achieves an insert and query throughput, confirming its practical advantages in a production-like environment.

VII. CONCLUSION

This paper introduced the Hypersistent Sketch, a unified framework that addresses the core challenge of item collision in single-structure persistence sketches. Its decoupled design features a Cold Filter for memory-efficient item separation and a Burst Filter to accelerate throughput. This framework can be prepended to any existing sketch, acting as a performance amplifier. Extensive experiments show our framework consistently enhances various state-of-the-art sketches (e.g., On-Off, Waving, P-Sketch), significantly improving accuracy while increasing insertion throughput by up to $3\times$ with no penalty to query speed. Successful deployment on Redis further proves its real-world applicability, establishing Hypersistent Sketch as a practical tool for persistence estimation tasks.

REFERENCES

- [1] L. Cao et al., "Hypersistent sketch: Enhanced persistence estimation via fast item separation," in *Proc. IEEE 41st Int. Conf. Data Eng.*, 2025, pp. 3030–3042.
- [2] H. Huang et al., "An efficient K-Persistent spread estimator for traffic measurement in high-speed networks," *IEEE/ACM Trans. Netw.*, vol. 28, no. 4, pp. 1463–1476, Aug. 2020.
- [3] Y.-E. Sun, H. Huang, S. Chen, Y. Zhou, K. Han, and W. Yang, "Privacy-preserving estimation of K-Persistent traffic in vehicular cyber-physical systems," *IEEE Internet Things J.*, vol. 6, no. 5, pp. 8296–8309, Oct. 2019.
- [4] H. Huang et al., "You can drop but you can't hide: K-persistent spread estimation in high-speed networks," in *Proc. IEEE Conf. Comput. Commun.*, 2018, pp. 1889–1897.
- [5] I. A. Elgendy, W. Zhang, Y.-C. Tian, and K. Li, "Resource allocation and computation offloading with data security for mobile edge computing," *Future Gener. Comput. Syst.*, vol. 100, pp. 531–541, 2019.
- [6] T. Yang, H. Zhang, D. Yang, Y. Huang, and X. Li, "Finding significant items in data streams," in *Proc. IEEE 35th Int. Conf. Data Eng.*, 2019, pp. 1394–1405.
- [7] S. Guha, J. Chandrashekar, and N. Taft, "How healthy are today's enterprise networks," in *Proc. ACM SIGCOMM Comput. Commun. Rev.*, 2008, pp. 145–150.
- [8] M. Karpapa and R. Pagh, "HyperLogLogLog: Cardinality estimation with one log more," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2022, pp. 753–761.
- [9] A. Alshamrani, S. Myneni, A. Chowdhary, and D. Huang, "A survey on advanced persistent threats: Techniques, solutions, challenges, and research opportunities," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 2, pp. 1851–1877, Second Quarter 2019.
- [10] L. Shang, D. Guo, Y. Ji, and Q. Li, "Discovering unknown advanced persistent threat using shared features mined by neural networks," *Comput. Netw.*, vol. 189, 2021, Art. no. 107937.

- [11] I. Ghafir et al., "Advanced persistent threat attack detection: An overview," *Int. J. Adv. Comput. Netw. Secur.*, vol. 4, no. 4, 2014, Art. no. 5054.
- [12] N. Tang, Q. Chen, and P. Mitra, "Graph stream summarization: From big bang to big crunch," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2016, pp. 1481–1496.
- [13] M. Li et al., "Seesaw counting filter: A dynamic filtering framework for vulnerable negative keys," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12987–13001, Dec. 2023.
- [14] A. Sharma, B. B. Gupta, A. K. Singh, and V. Saraswat, "Advanced persistent threats (APT): Evolution, anatomy, attribution and countermeasures," *J. Ambient Intell. Humanized Comput.*, vol. 14, no. 7, pp. 9355–9381, 2023.
- [15] Z. Fan, Z. Hu, Y. Wu, J. Guo, W. Liu, and T. Yang, "PISketch: Finding persistent and infrequent flows," in *Proc. ACM SIGCOMM Comput. Commun. Rev.*, 2022, pp. 8–14.
- [16] L. Chen, H. Dai, L. Meng, and J. Yu, "Finding needles in a hay stream: On persistent item lookup in data streams," *Comput. Netw.*, vol. 181, 2020, Art. no. 107518.
- [17] H. Dai, M. Li, A. X. Liu, J. Zheng, and G. Chen, "Finding persistent items in distributed datasets," *IEEE/ACM Trans. Netw.*, vol. 28, no. 1, pp. 1–14, Feb. 2020.
- [18] H. Dai, M. Shahzad, A. X. Liu, M. Li, Y. Zhong, and G. Chen, "Identifying and estimating persistent items in data streams," *IEEE/ACM Trans. Netw.*, vol. 26, no. 6, pp. 2429–2442, Dec. 2018.
- [19] W. Zhang et al., "Secure and optimized load balancing for multitier IoT and edge-cloud computing systems," *IEEE Internet Things J.*, vol. 8, no. 10, pp. 8119–8132, May 2021.
- [20] M. Greenwald and S. Khanna, "Space-efficient online computation of quantile summaries," *ACM SIGMOD Rec.*, vol. 30, no. 2, pp. 58–66, 2001.
- [21] D. M. Powers, "Applications and explanations of Zipf's law," in *Proc. Conf. Empir. Methods Natural Lang. Process., Comput. Natural Lang. Learn.*, 1998, pp. 151–160.
- [22] T. Benson, A. Akella, and D. A. Maltz, "Network traffic characteristics of data centers in the wild," in *Proc. ACM SIGCOMM Comput. Commun. Rev.*, 2010, pp. 267–280.
- [23] H. Yang, H. He, W. Zhang, and X. Cao, "FedSteg: A federated transfer learning framework for secure image steganalysis," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 1084–1094, Second Quarter 2021.
- [24] Z. Fan et al., "OneSketch: A generic and accurate sketch for data streams," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12887–12901, Dec. 2023.
- [25] H. Li et al., "Stingy sketch: A sketch framework for accurate and fast frequency estimation," *Proc. ACM VLDB*, vol. 15, pp. 1426–1438, 2022.
- [26] H. Huang et al., "Memory-efficient and flexible detection of heavy hitters in high-speed networks," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2024, pp. 1–24.
- [27] C. Estan, G. Varghese, and M. Fisk, "Bitmap algorithms for counting active flows on high speed links," in *Proc. 3rd ACM SIGCOMM Conf. Internet Measur.*, 2003, pp. 153–166.
- [28] L. Wang, G. Luo, K. Yi, and G. Cormode, "Quantiles over data streams: An experimental study," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2013, pp. 737–748.
- [29] R. Yadav, W. Zhang, O. Kaiwartya, H. Song, and S. Yu, "Energy-latency tradeoff for dynamic computation offloading in vehicular fog computing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 14198–14211, Dec. 2020.
- [30] Q. Zhou et al., "CBA sketch: A sketching algorithm mining persistent batches in data streams," in *Proc. Int. Conf. Algorithms Architectures Parallel Process.*, 2023, pp. 114–132.
- [31] T. Yang, S. Gao, Z. Sun, Y. Wang, Y. Shen, and X. Li, "Diamond sketch: Accurate per-flow measurement for big streaming data," *IEEE Trans. Parallel Distrib. Syst.*, vol. 30, no. 12, pp. 2650–2662, Dec. 2019.
- [32] C. Guo et al., "Pingmesh: A large-scale system for data center network latency measurement and analysis," in *Proc. ACM Conf. Special Int. Group Data Commun.*, 2015, pp. 139–152.
- [33] Y. Zhu et al., "Packet-level telemetry in large datacenter networks," in *Proc. ACM Conf. Special Int. Group Data Commun.*, 2015, pp. 479–491.
- [34] Y. Zhang et al., "On-off sketch: A fast and accurate sketch on persistence," *Proc. VLDB Endowment*, vol. 13, no. 11, pp. 2372–2385, 2020.
- [35] H. Dai, M. Shahzad, and A. X. Liu, "Finding persistent items in data streams," *Proc. VLDB Endowment*, vol. 10, no. 4, pp. 289–300, 2016.
- [36] Y. Zhou et al., "Cold filter: A meta-framework for faster and more accurate stream processing," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2018, pp. 741–756.
- [37] J. Tan and J. Wang, "Detecting advanced persistent threats based on entropy and support vector machine," in *Proc. Int. Conf. Algorithms Archit. Parallel Process.*, 2018, pp. 153–165.
- [38] Q. Xiao, Y. Qiao, M. Zhen, and S. Chen, "Estimating the persistent spreads in high-speed networks," in *Proc. IEEE 22nd Int. Conf. Netw. Protoc.*, 2014, pp. 131–142.
- [39] S. A. Singh and S. Tirthapura, "Monitoring persistent items in the union of distributed streams," *J. Parallel Distrib. Comput.*, vol. 74, no. 11, pp. 3115–3127, 2014.
- [40] Y. Peng, J. Guo, F. Li, W. Qian, and A. Zhou, "Persistent bloom filter: Membership testing for the entire history," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2018, pp. 1037–1052.
- [41] Y.-E. Sun, H. Huang, and S. Chen, H. Xu, K. Han, and Y. Zhou, "Persistent traffic measurement through vehicle-to-infrastructure communications in cyber-physical road systems," *IEEE Trans. Mobile Comput.*, vol. 18, no. 7, pp. 1616–1630, Jul. 2019.
- [42] Q. Shi et al., "Cuckoo counter: Adaptive structure of counters for accurate frequency and top-k estimation," *IEEE/ACM Trans. Netw.*, vol. 31, no. 4, pp. 1854–1869, Aug. 2023.
- [43] Q. Shi et al., "BitMatcher: Bit-level counter adjustment for sketches," in *Proc. IEEE 40th Int. Conf. Data Eng.*, 2024, pp. 4815–4827.
- [44] L. Cao et al., "Bubble sketch: A high-performance and memory-efficient sketch for finding top-k items in data streams," in *Proc. ACM Conf. Inf. Knowl. Manage.*, 2024, pp. 3653–3657.
- [45] T. Yang et al., "Elastic sketch: Adaptive and fast network-wide measurements," in *Proc. ACM SIGCOMM Comput. Commun. Rev.*, 2018, pp. 561–575.
- [46] D. Ting, "Count-min: Optimal estimation and tight error bounds using empirical error distributions," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 2319–2328.
- [47] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Commun. ACM*, vol. 13, no. 7, pp. 422–426, 1970.
- [48] J. Li et al., "Wavingsketch: An unbiased and generic sketch for finding top-k items in data streams," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 1574–1584.
- [49] W. Li and P. Patras, "P-sketch: A fast and accurate sketch for persistent item lookup," *IEEE/ACM Trans. Netw.*, vol. 32, no. 2, pp. 987–1002, Apr. 2024.
- [50] W. Li and P. Patras, "Tight-sketch: A high-performance sketch for heavy item-oriented data stream mining with limited memory size," in *Proc. ACM Conf. Inf. Knowl. Manage.*, 2023, pp. 1328–1337.
- [51] W. Li et al., "Stable-sketch: A versatile sketch for accurate, fast, web-scale data stream processing," in *Proc. ACM Int. Conf. World Wide Web*, 2024, pp. 4227–4238.
- [52] "Our GitHub," 2026. [Online]. Available: <https://github.com/wenjunpa per/HypersistentSketch>
- [53] C. Estan and G. Varghese, "New directions in traffic measurement and accounting," in *Proc. ACM SIGCOMM Comput. Commun. Rev.*, 2002, pp. 323–336.
- [54] "Our Website," 2026. [Online]. Available: <https://turbonet.team/HypersistentSketch>
- [55] MAWI Working Group, "MAWI working group traffic archive," 2024. Accessed: Sep. 30, 2024. [Online]. Available: <http://mawi.wide.ad.jp/mawi/>
- [56] Y. Zhang et al., "On-off sketch: A fast and accurate sketch on persistence," *Proc. VLDB Endowment*, vol. 14, no. 2, pp. 128–140, 2020.
- [57] B. Lahiri, J. Chandrashekar, and S. Tirthapura, "Space-efficient tracking of persistent items in a massive data stream," in *Proc. 5th ACM Int. Conf. Distrib. Event-Based Syst.*, 2011, pp. 255–266.
- [58] "I. S. Documentation," 2024. [Online]. Available: <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html>
- [59] "Redis website," 2022. Accessed: 2022. [Online]. Available: <https://redis.io>
- [60] "Redisbloom," 2022. Accessed: 2022. [Online]. Available: <https://github.com/RedisBloom/RedisBloom>



Qilong Shi received the BSc degree in computer science from Peking University, in 2022. He is currently working toward the PhD degree with the Department of Computer Science and Technology, Tsinghua University. He has authored or coauthored several papers as the first author in SIGKDD, ICDE, and TON. His research interests include sketch, massive data stream processing, and network measurement.



Lu Cao received the MS degree from the College of Computer, National University of Defense Technology, Changsha, China. She is currently working toward the PhD degree with the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, China. Her research interests include sketches, network measurement, and network systems.



Tong Yang (Member, IEEE) received the PhD degree in computer science from Tsinghua University, in 2013. He visited the Institute of Computing Technology, Chinese Academy of Sciences. He is currently an associate professor with the Department of Computer Science and Technology, Peking University. He has authored or coauthored more than 20 papers in SIGCOMM, SIGKDD, SIGMOD, and NSDI. His research interests include network measurements, sketches, IP lookups, bloom filters and KV stores.



Weiqiang Xiao is currently working toward the MS degree in computer science and technology at the Harbin Institute of Technology, Shenzhen. His research focuses on hardware-software co-design of sketch algorithms and network measurement.



Zhijun Li is a professor with the Harbin Institute of Technology, Harbin, China, and a special professor with Soochow University, Suzhou, China. He has authored or coauthored more than 40 papers in IEEE/ACM TON, *IEEE Transactions on Mobile Computing*, MOBICOM, MOBISYS, SENSYS, INFOCOM and other journals/conferences, and has received the best paper (first author) in MOBICOM. His research interests include IoT, AI, and operating systems.



Nianfu Wang received the master's degree from the Harbin Institute of Technology. He is currently working toward the PhD degree with the Harbin Institute of Technology, Shenzhen. His research interests include the Internet of Things and Artificial Intelligence, edge intelligence, and multi-device collaboration.



Weizhe Zhang (Senior Member, IEEE) received the PhD degree from the Harbin Institute of Technology, in 2006. He is currently a professor with the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, China, and the Director with Department of New Networks, Peng Cheng Laboratory, Shenzhen, China. His research interests include cyberspace security, cloud computing, and high-performance computing. He is a lifetime member of the ACM.



Wenjun Li received the PhD degree from Peking University, in 2020. From 2021 to 2023, he was a postdoctoral fellow with Harvard University. He is currently an associate researcher with Pengcheng Laboratory, and a PhD supervisor with the Southern University of Science and Technology. His research interests include programmable network, network telemetry, and network algorithms.



Mingwei Xu received the BSc and PhD degrees from Tsinghua University. He is currently a full professor with the Department of Computer Science, Tsinghua University. His research interests include computer network architecture, high-speed router architecture, and network security.